



Lecture 2: Word Embeddings



NYU

Greg Durrett

Administrative details and recap

Neural Networks and Backpropagation

Word Embeddings

Skip-gram

GloVe

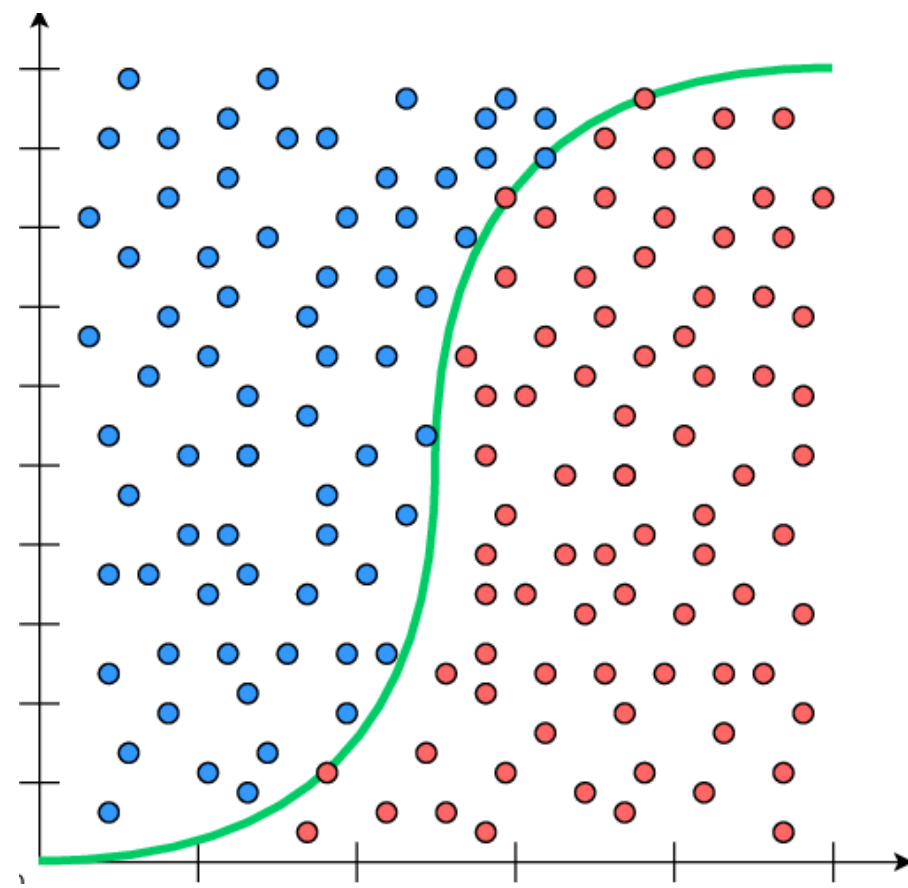
Evaluating Word Embeddings

Recap

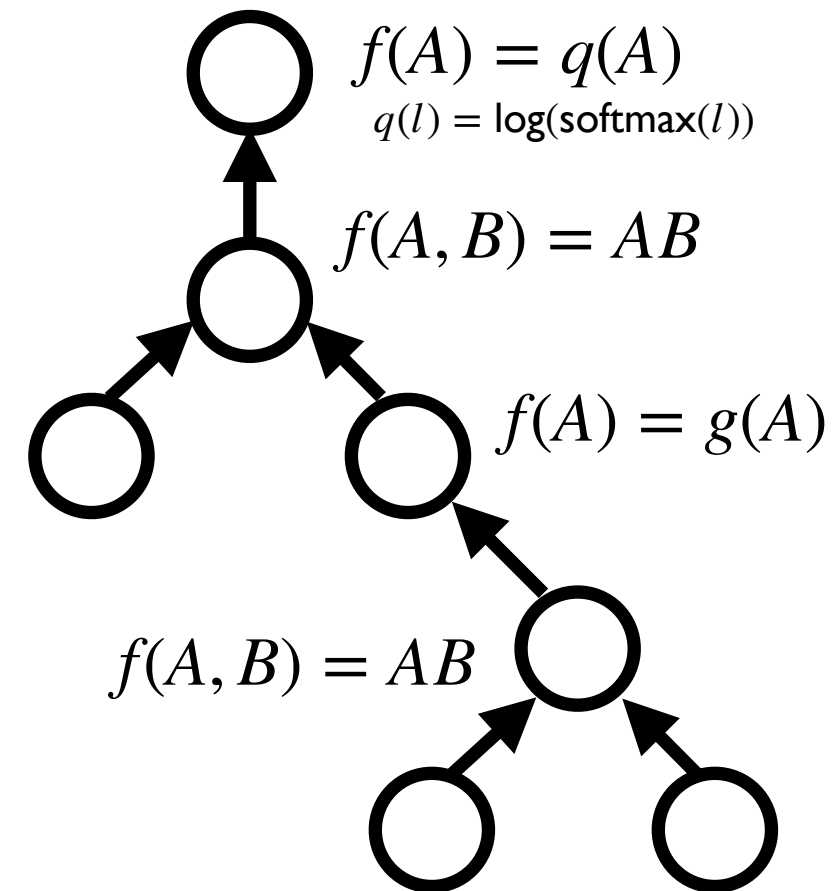
Announcements

- ▶ A1 due next week
- ▶ Quiz 1 in class next week; covers content from week 1 and week 2
 - ▶ Style of problems: see exams at <https://gregdurrett.github.io/courses/online-course/materials.html>
 - ▶ No note sheet allowed. Problems will stick close to A1 content

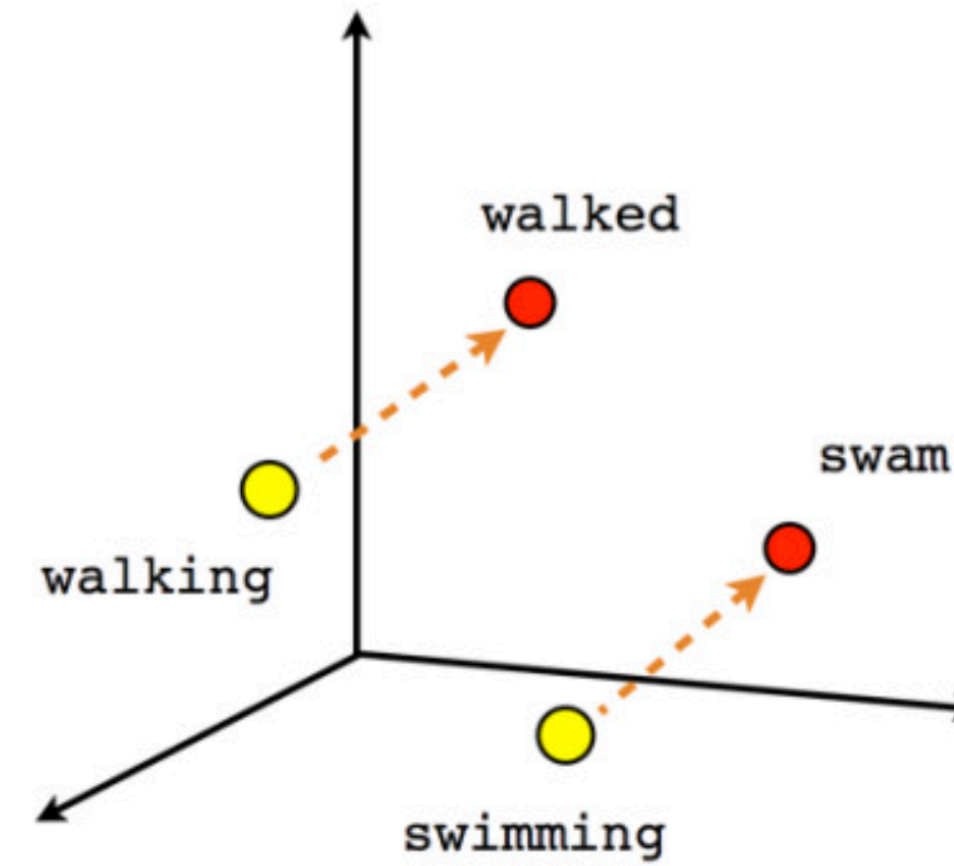
Overview



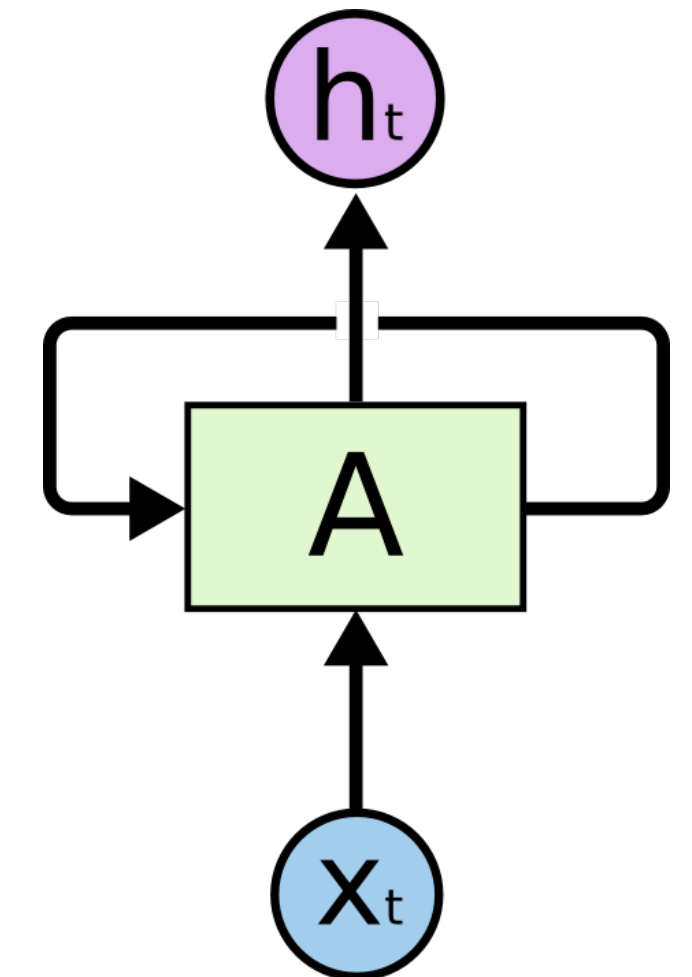
Logistic
Regression



Neural
Networks

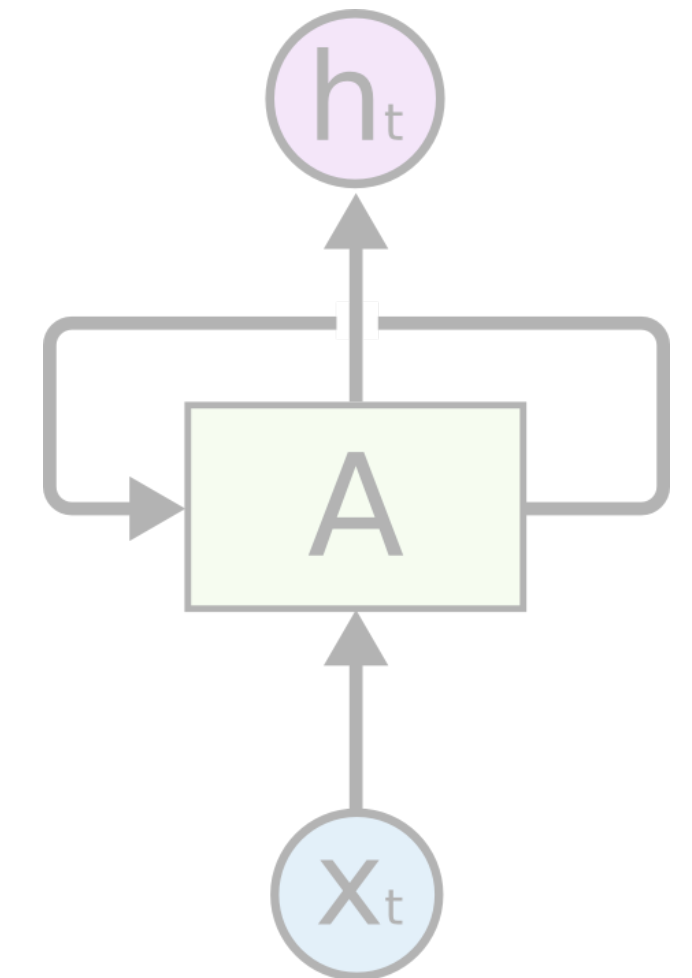
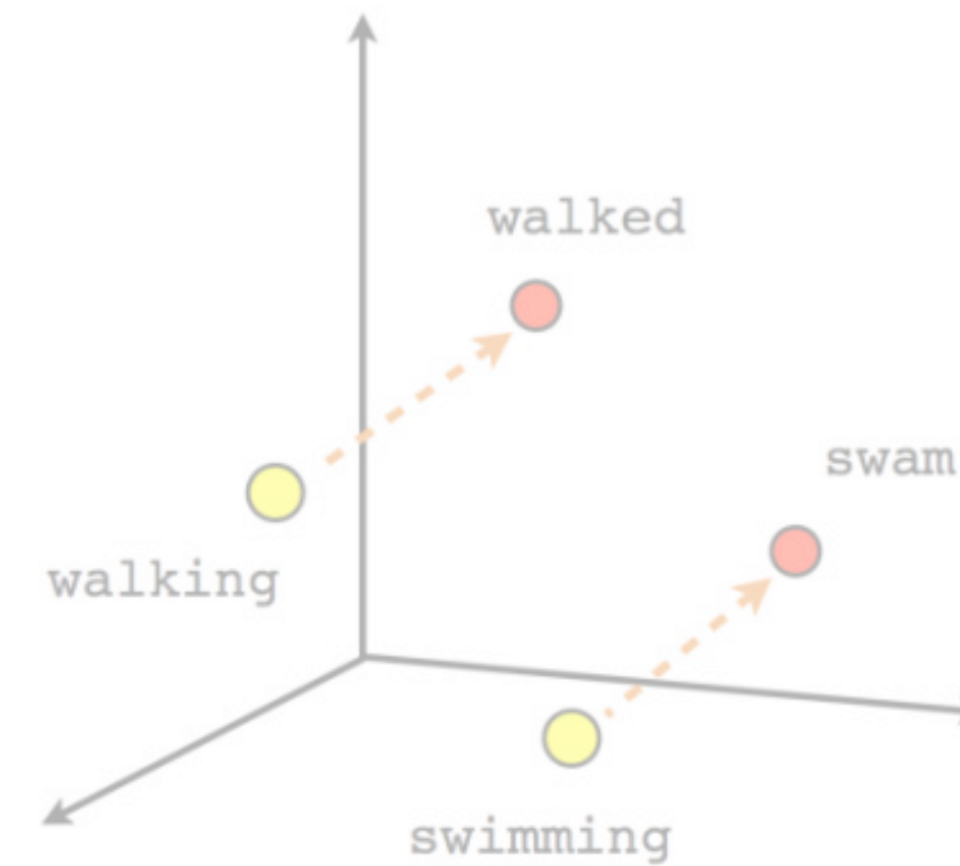
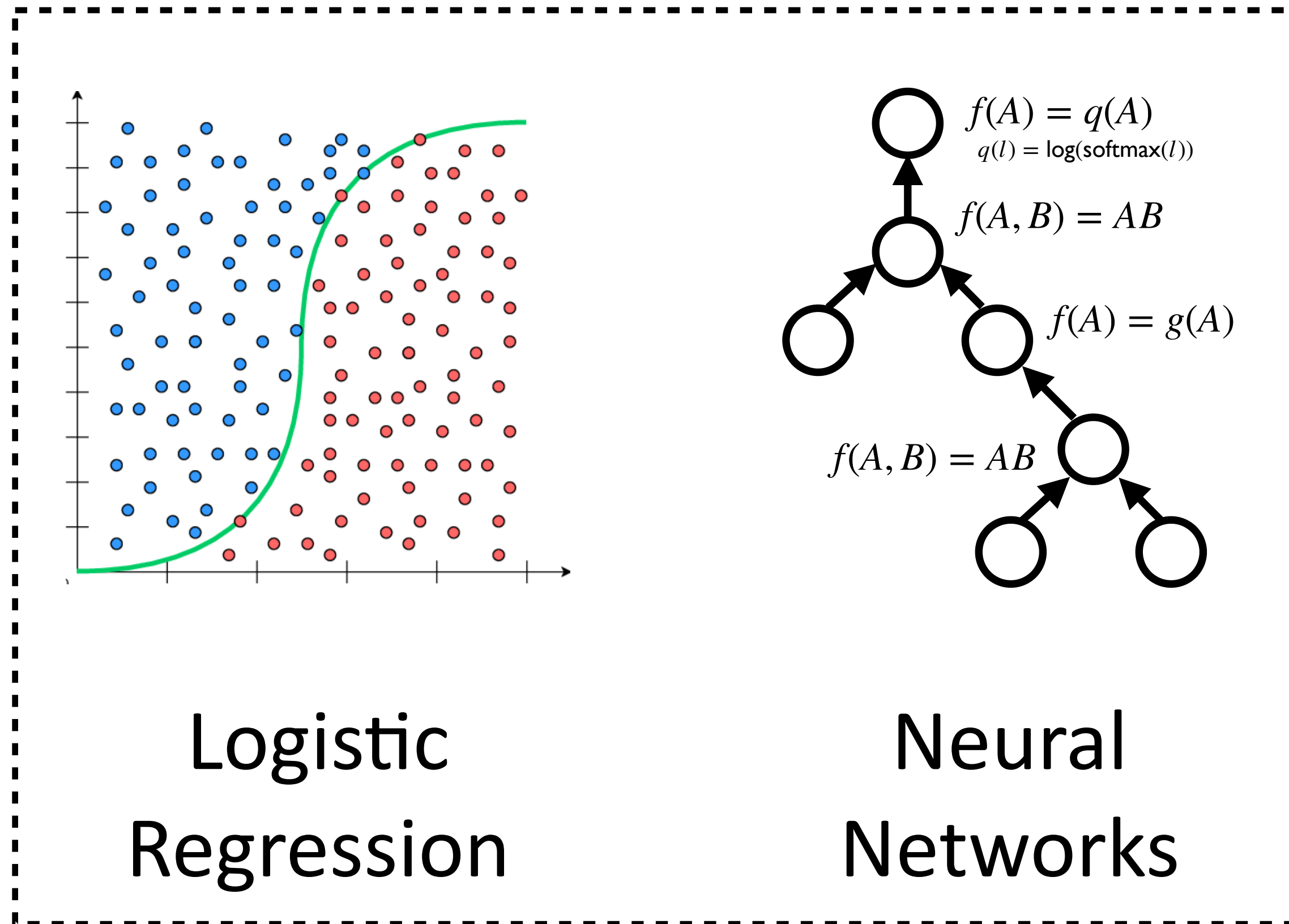


Word
Embeddings



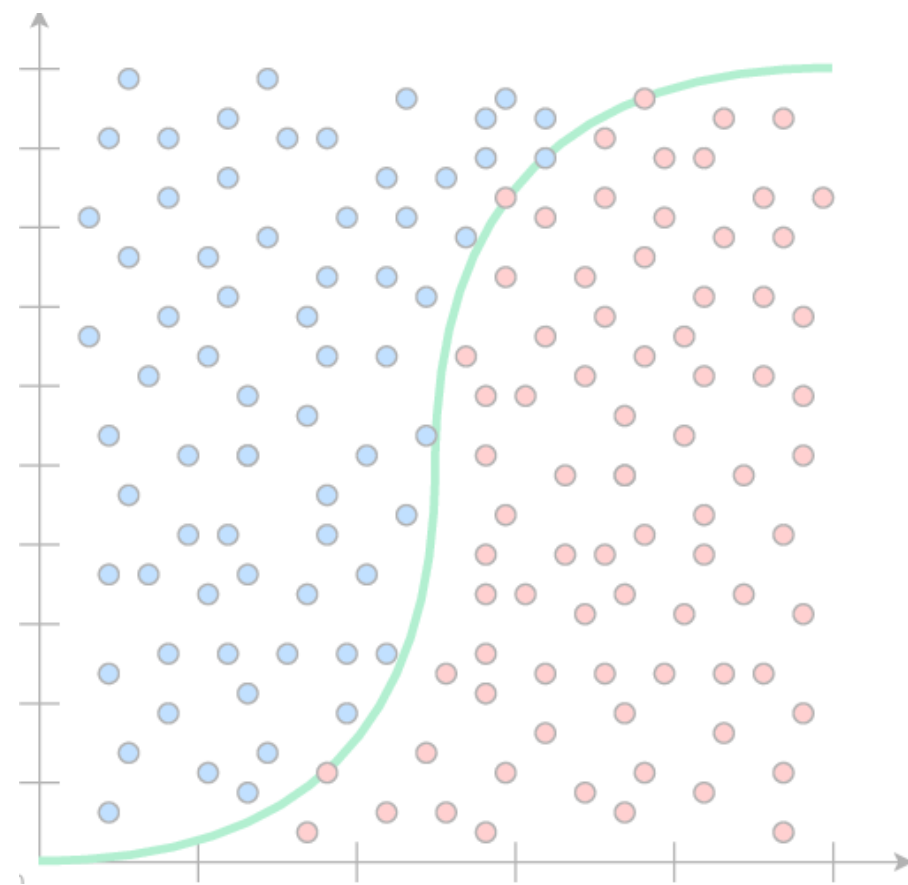
Sequence
Models

Overview

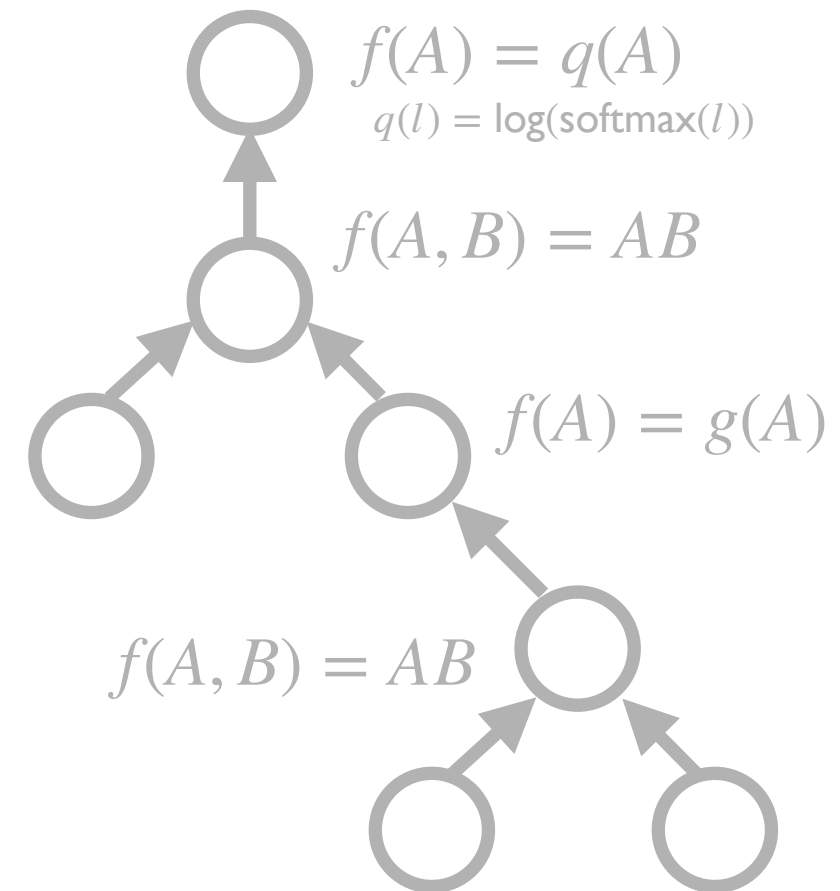


Review

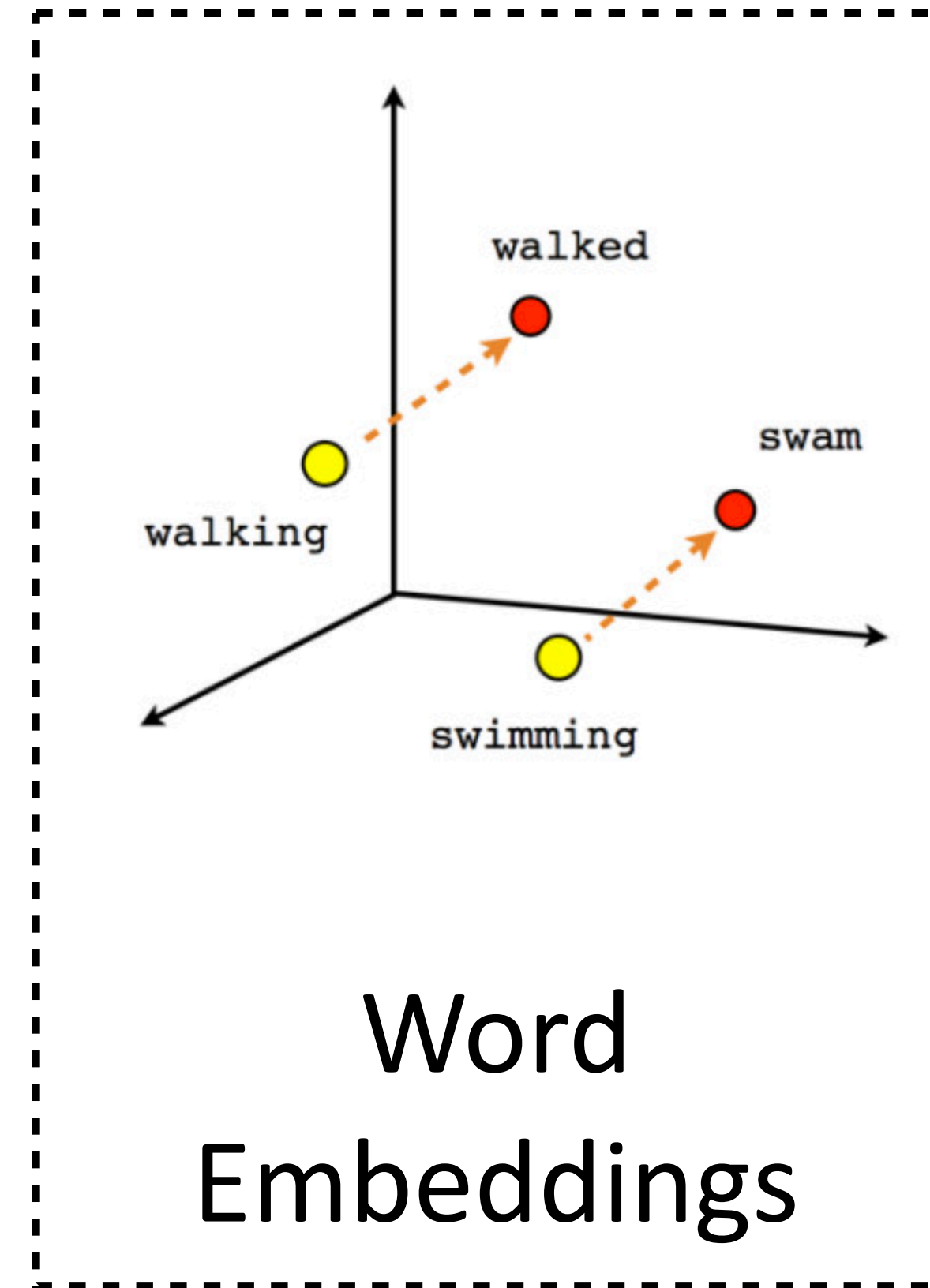
Overview



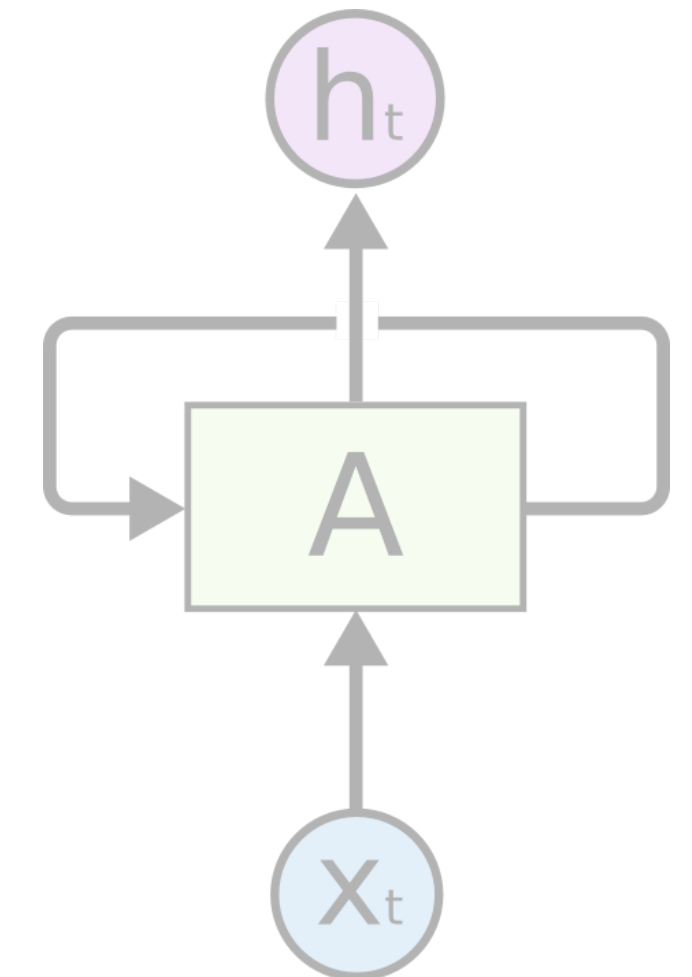
Logistic
Regression



Neural
Networks

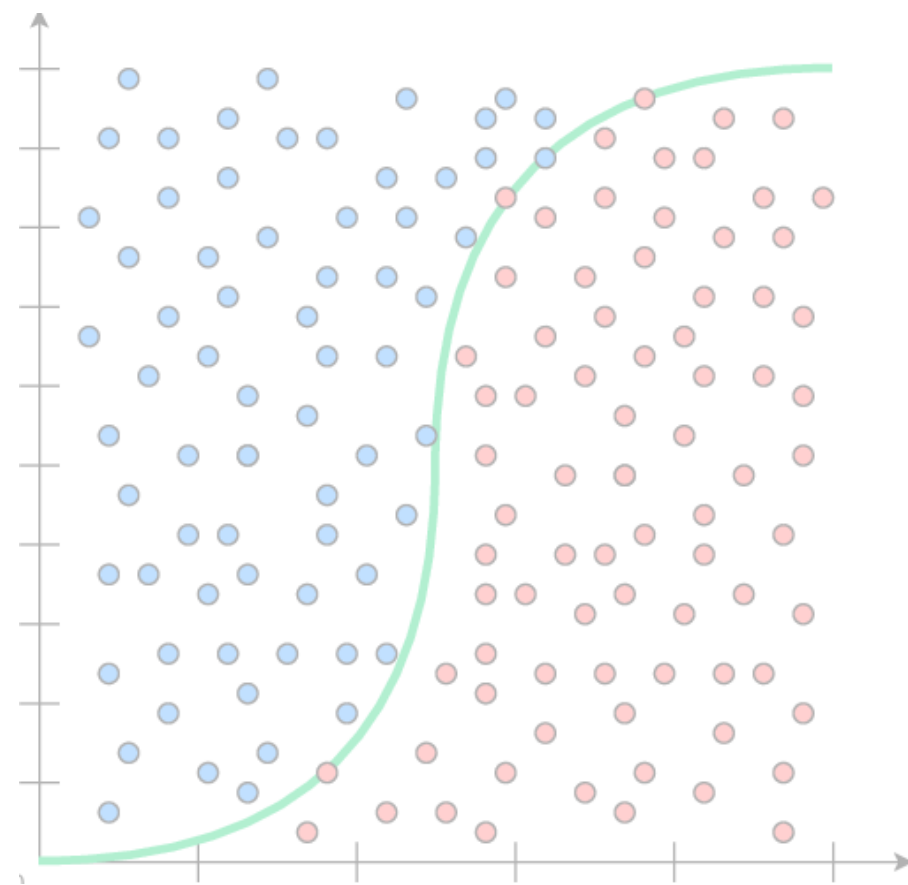


Word
Embeddings

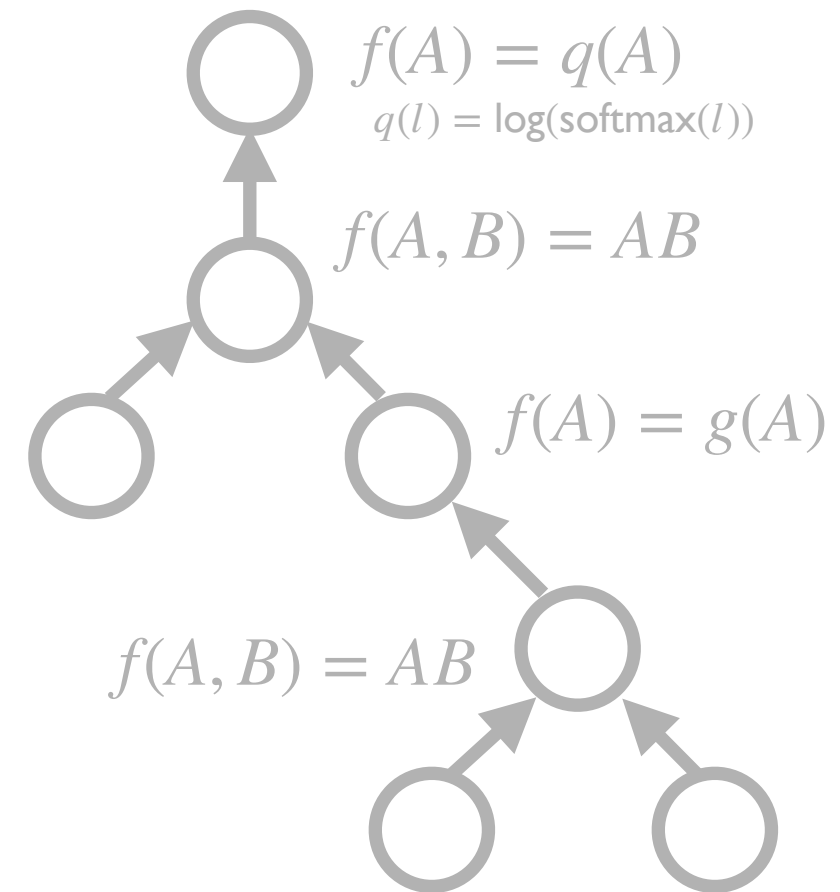


Main Content for Today

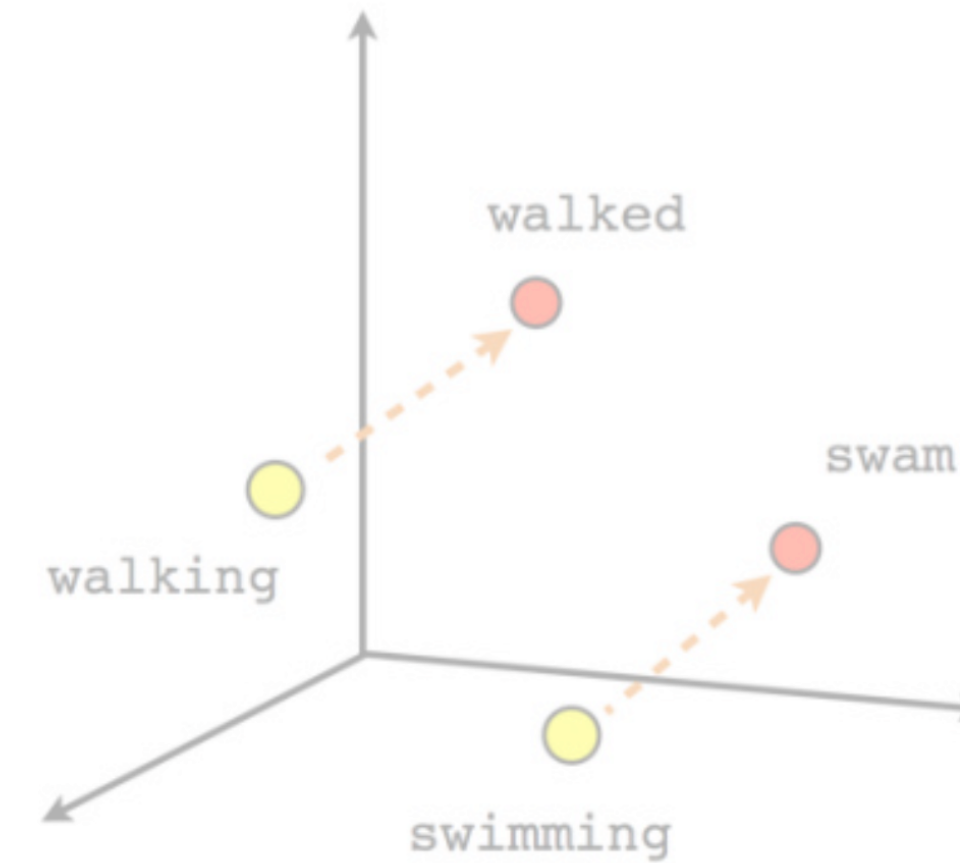
Overview



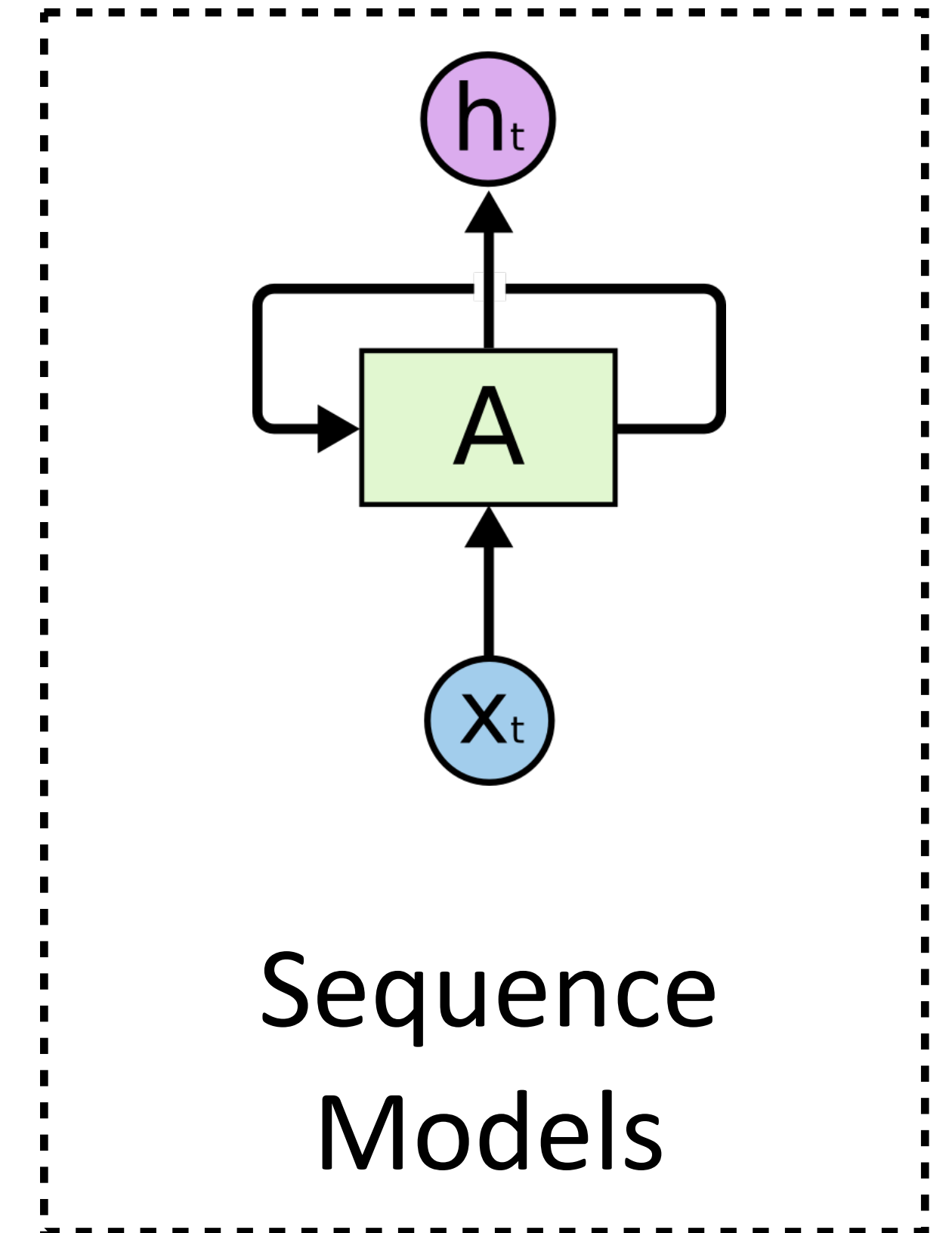
Logistic
Regression



Neural
Networks



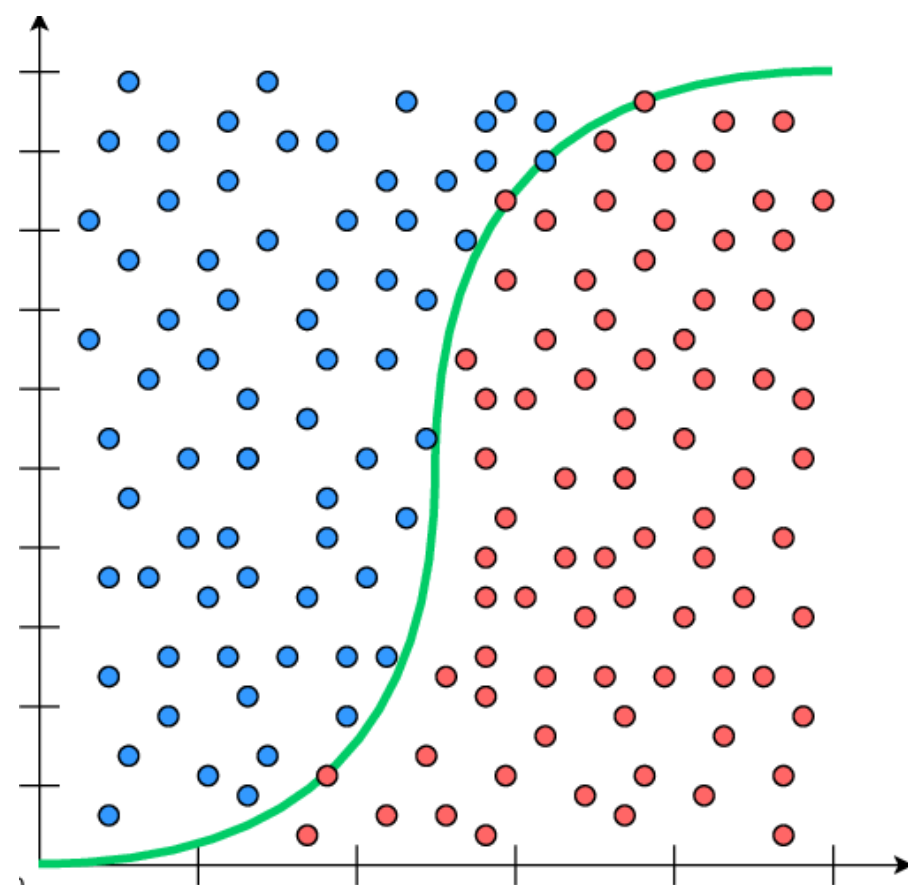
Word
Embeddings



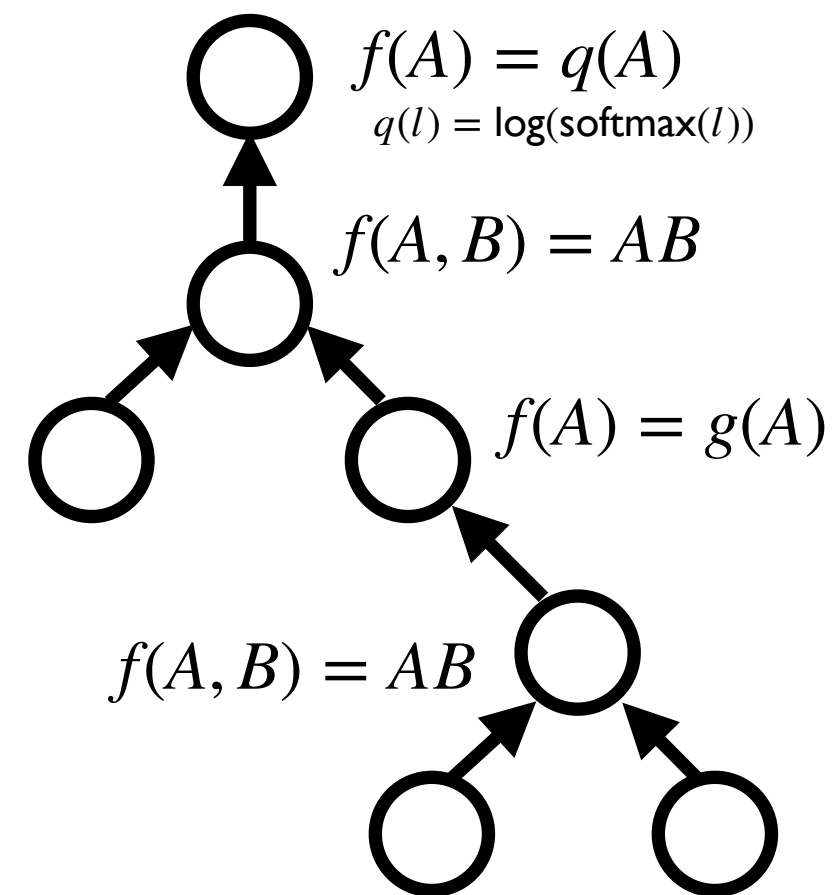
Sequence
Models

Next Week

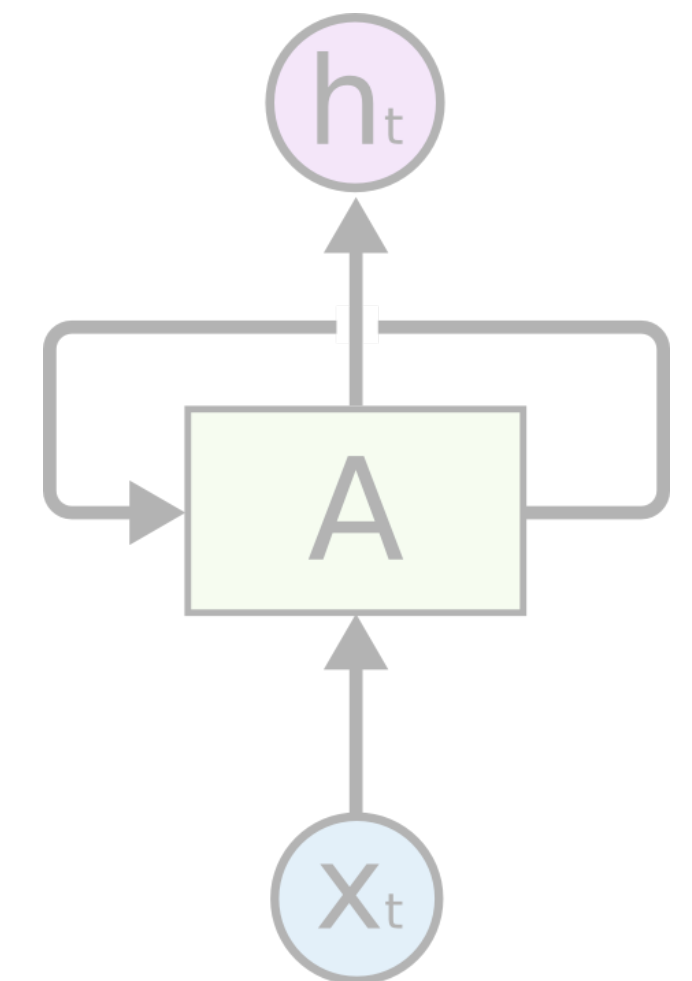
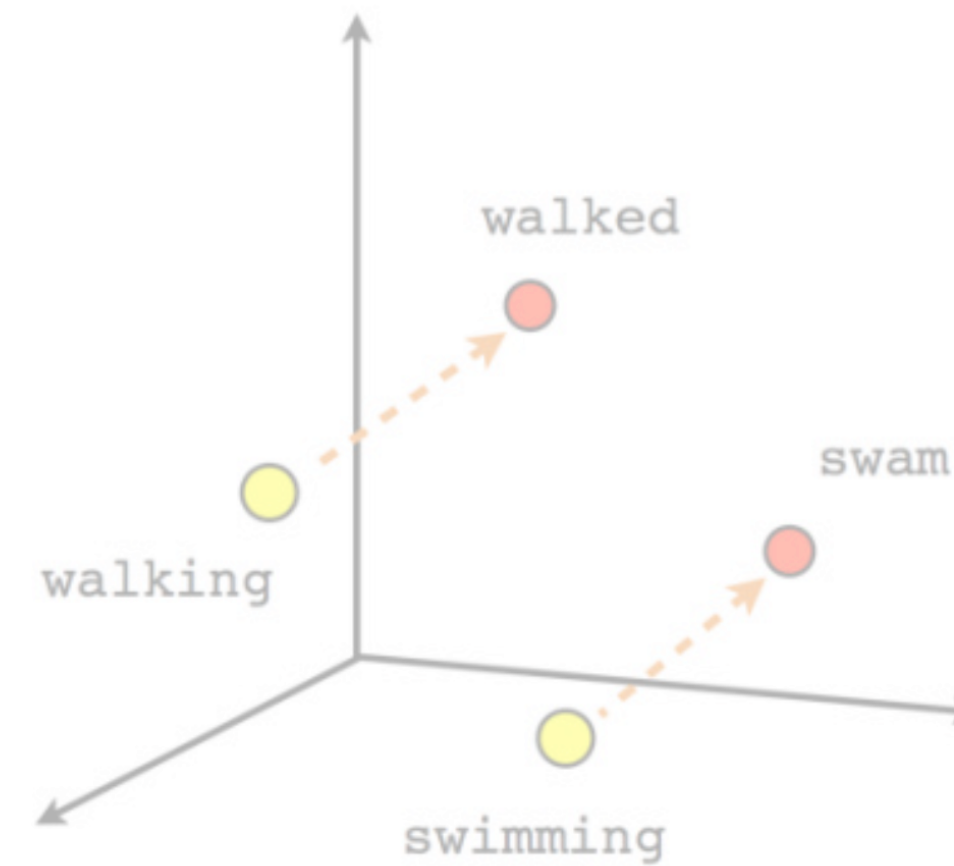
Overview



Logistic
Regression



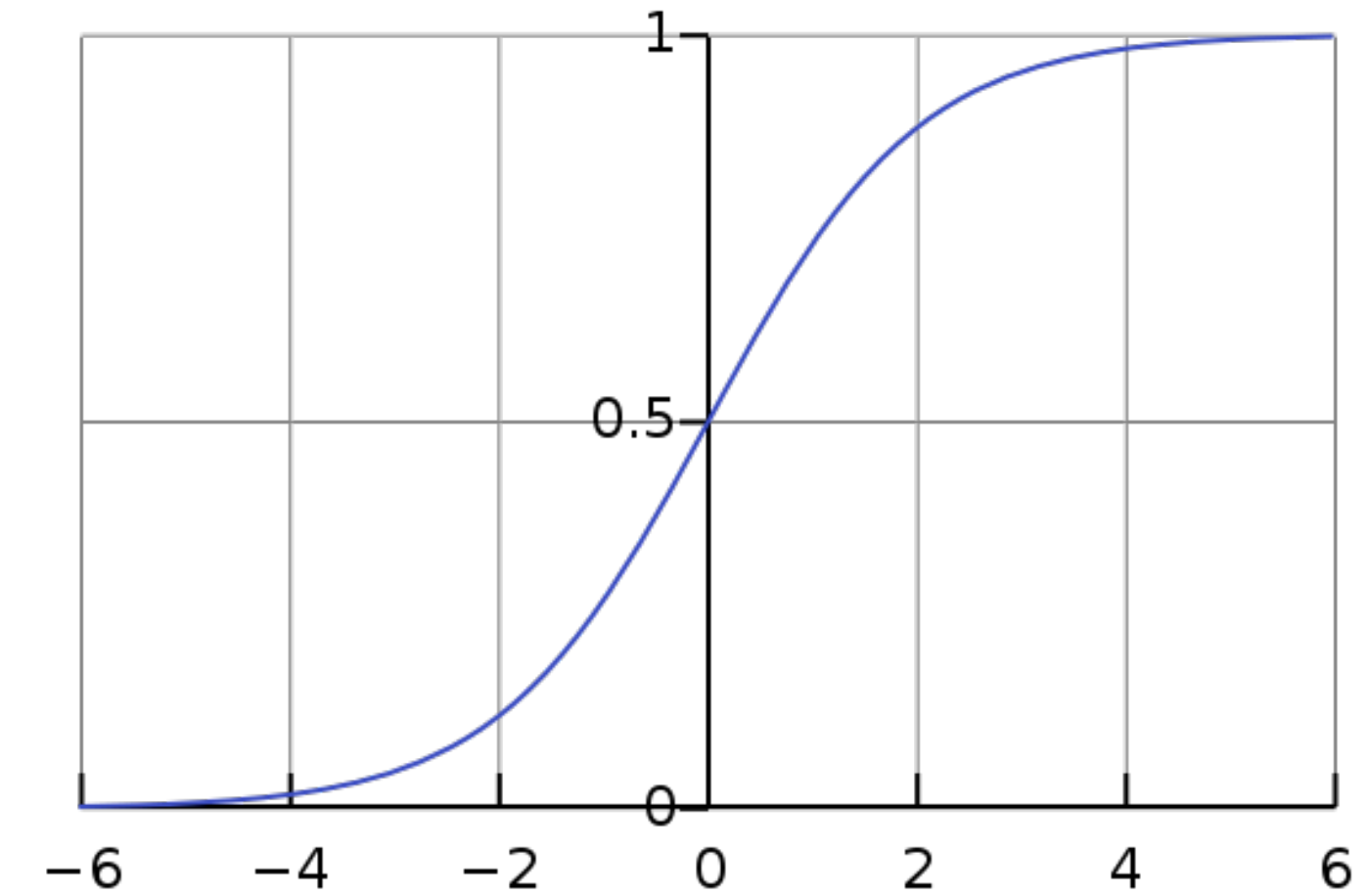
Neural
Networks



Recap: Logistic Regression

$$P(y = +|x) = \text{logistic}(w^\top x)$$

$$P(y = +|x) = \frac{\exp(\sum_{i=1}^n w_i x_i)}{1 + \exp(\sum_{i=1}^n w_i x_i)}$$



- To learn weights: maximize discriminative log likelihood of data ($\log P(y|x)$)

$$\mathcal{L}(\{x_j, y_j\}_{j=1, \dots, n}) = \sum_j \log P(y_j | x_j) \quad \text{corpus-level LL}$$

Recap: Feedforward NN for Multiclass Classification

$$P(y|\mathbf{x}) = \frac{\exp(\mathbf{w}_y^\top f(\mathbf{x}))}{\sum_{y' \in \mathcal{Y}} \exp(\mathbf{w}_{y'}^\top f(\mathbf{x}))}$$

- ▶ Single scalar probability

$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W f(\mathbf{x}))$$

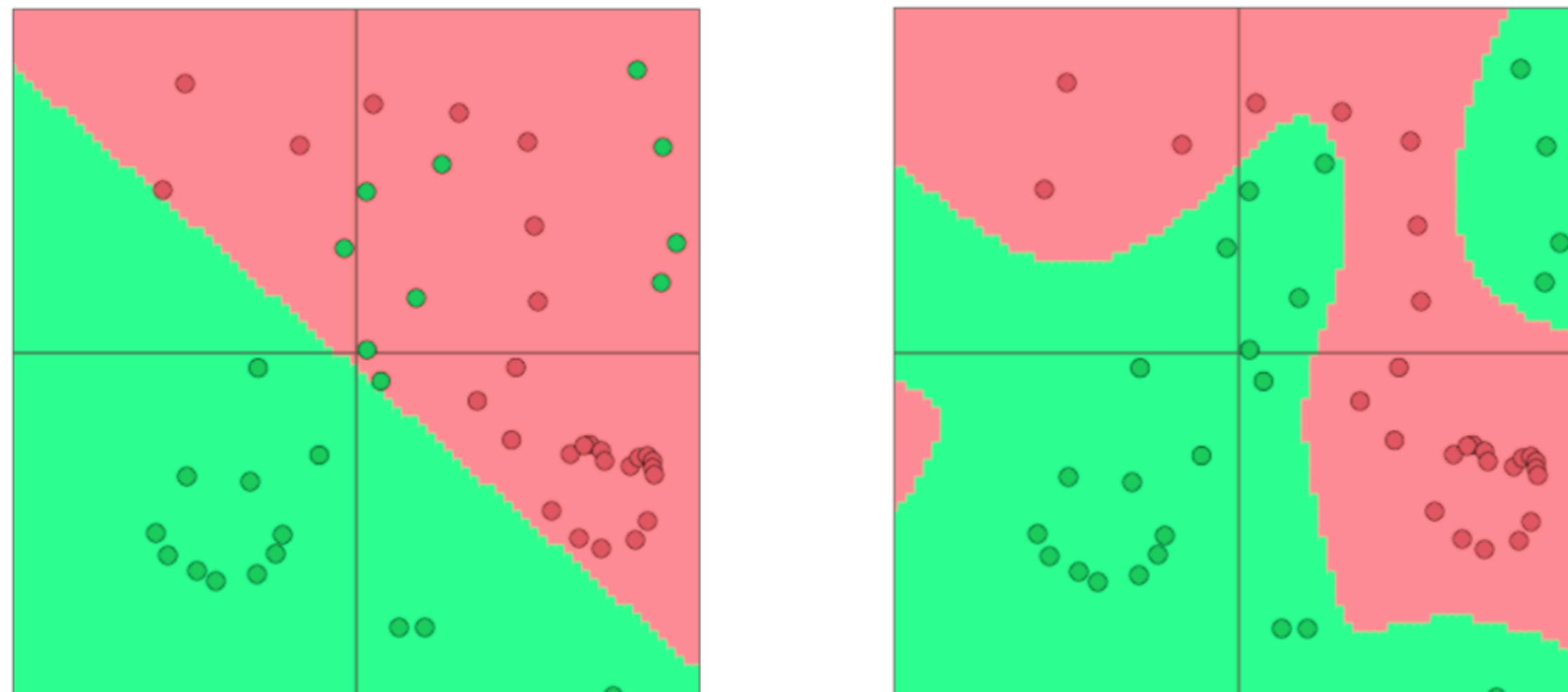
- ▶ Weight vector per class;
 W is [num classes x num feats]

$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W g(V f(\mathbf{x})))$$

- ▶ Neural network with one hidden layer

Recap: The “Promise”

- ▶ Most ML works with human-designed feature representations
- ▶ Representation Learning: automatically learn good features and representations
- ▶ Deep Learning: attempts to learn multilevel of representation of increasing complexity / abstraction



Recap: The “Promise”

- ▶ Representation Learning: automatically learn good features and representations

Multiclass logistic regression:

$$P(\mathbf{y} | \mathbf{x}) = \text{softmax}(w \cdot \phi(x, y))$$

You design the feature vector

NNs:

$$P(\mathbf{y} | \mathbf{x}) = \text{softmax}(Wg(H^1 f(\mathbf{x})))$$

$$P(\mathbf{y} | \mathbf{x}) = \text{softmax}(Wg(H^2(g(H^1 f(\mathbf{x}))))$$

Feature representations are
“learned” through hidden
layers

Administrative details and recap

Neural Networks and Backpropagation

Word Embeddings

Skip-gram

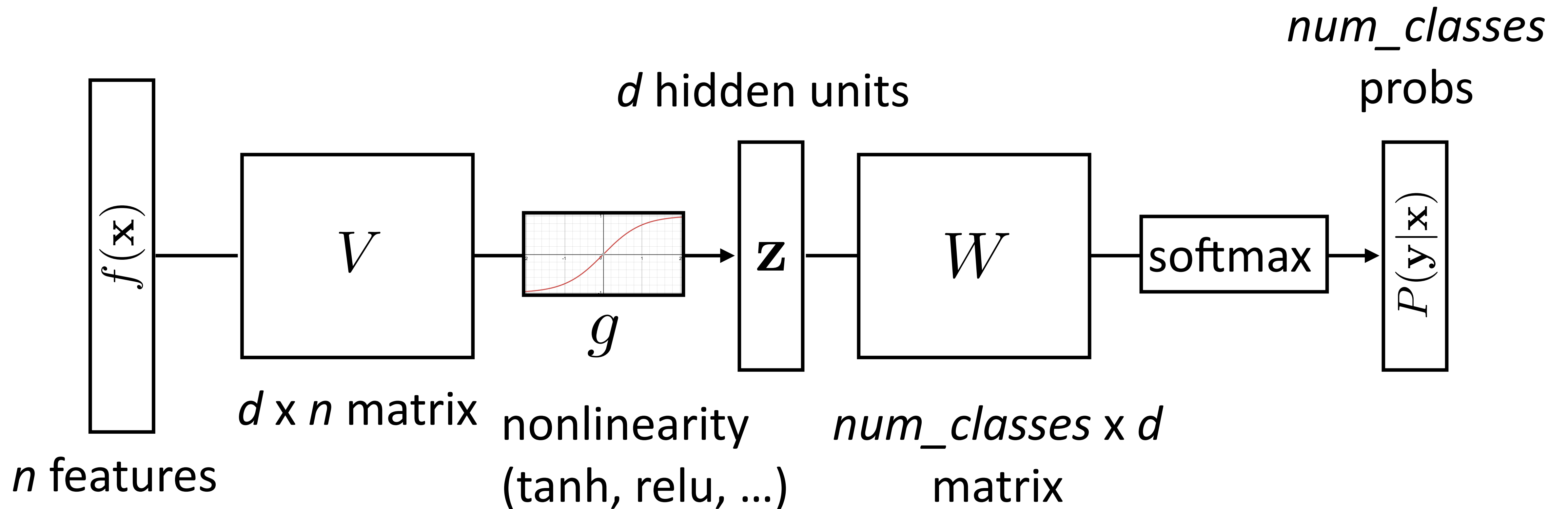
GloVe

Evaluating Word Embeddings

Neural Networks and Backpropagation

Neural Networks for Classification

$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W g(V f(\mathbf{x})))$$



Learning Objective

- ▶ We will use the same log likelihood objective we used for log linear models for classification tasks

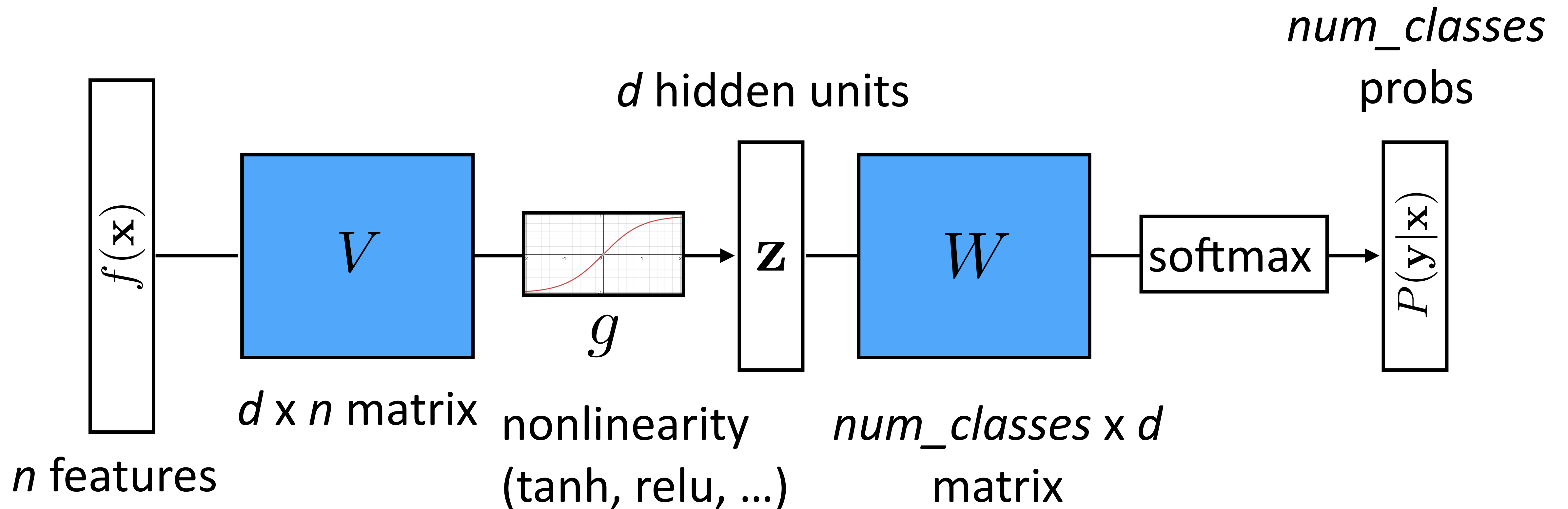
$$\log \prod_{i=1}^N p(y^i | x^i; w) = \sum_{i=1}^N \log(p(y^i | x^i; w))$$

- ▶ How do we compute the derivative when there are multiple layers of parameters?

$$L(w) = - \sum_{i=1}^N \log(p(y^i | x^i, w))$$

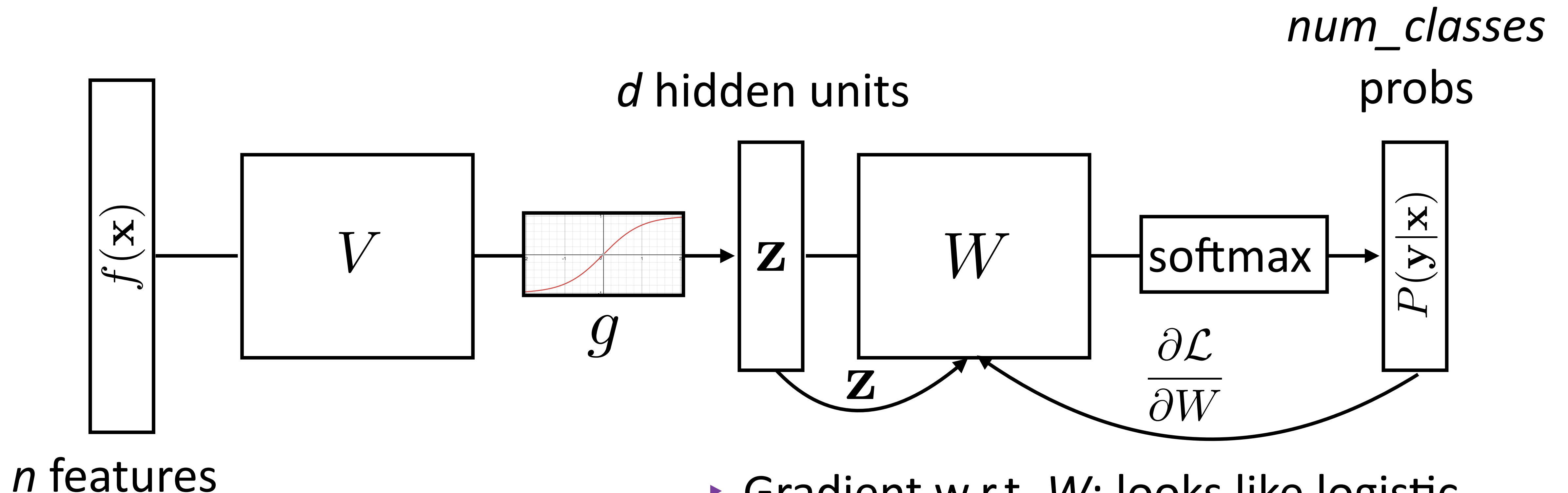
Neural Networks for Classification

$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W g(V f(\mathbf{x})))$$



Computing Gradients

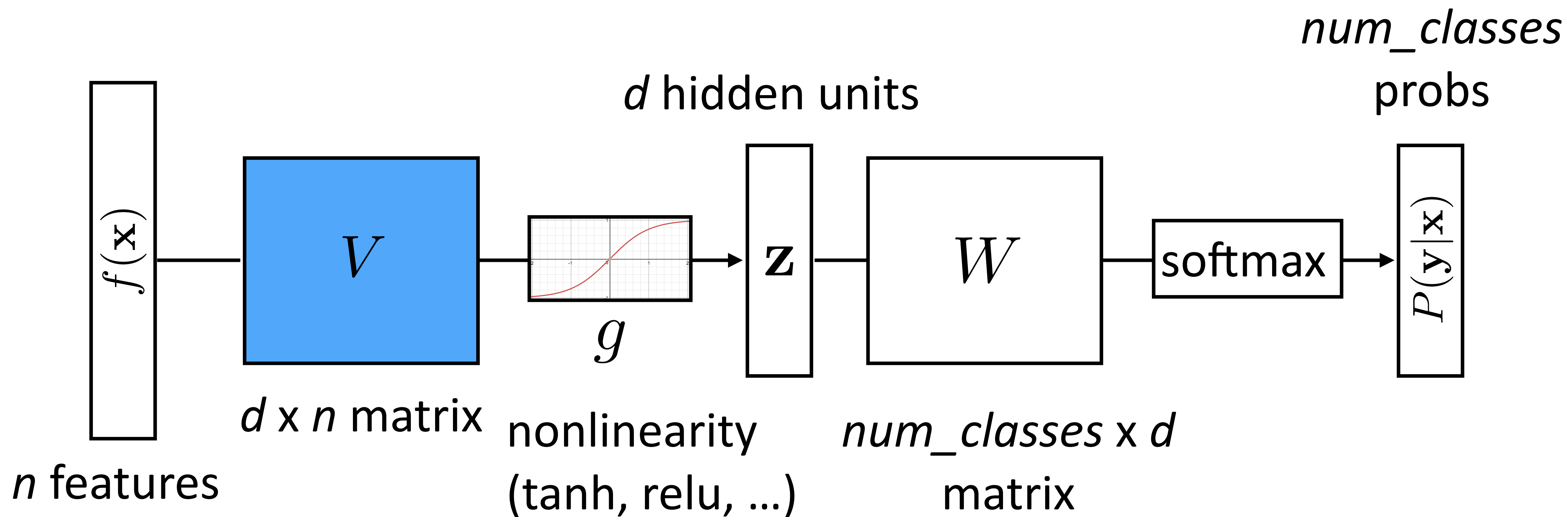
$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(Wg(Vf(\mathbf{x})))$$



- Gradient w.r.t. W : looks like logistic regression, can be computed treating $\mathbf{z}$ as the input features

Neural Networks for Classification

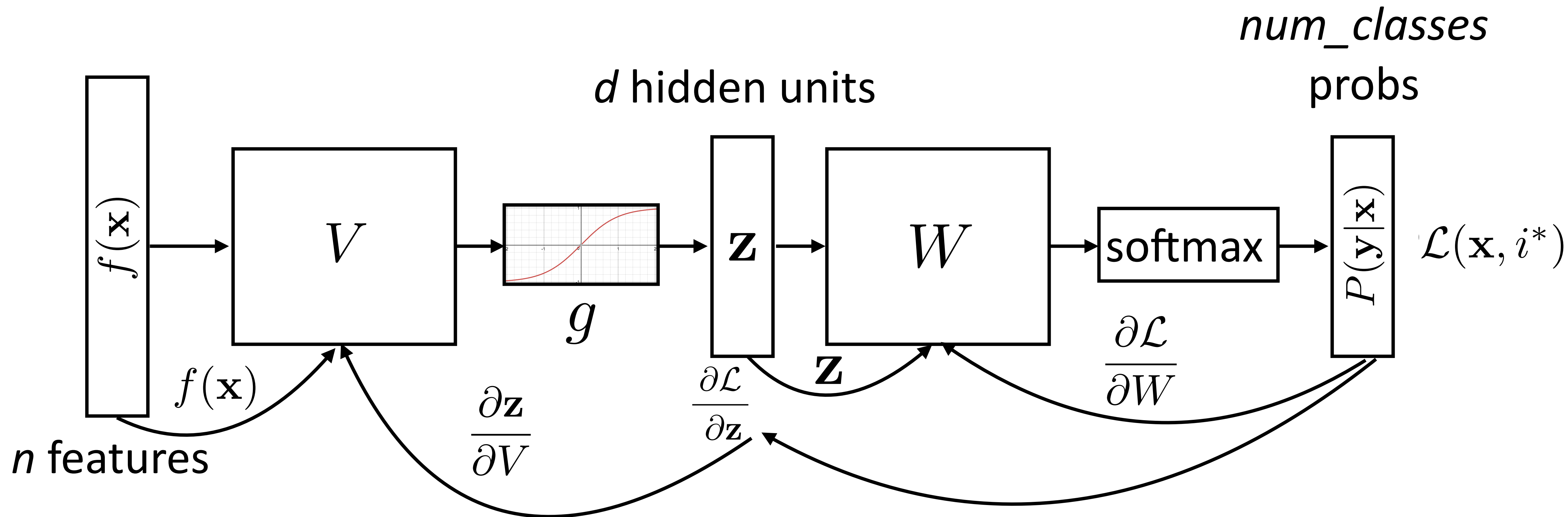
$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W g(V f(\mathbf{x})))$$



- How should we compute the gradient for intermediate parameters?

Computing Gradients

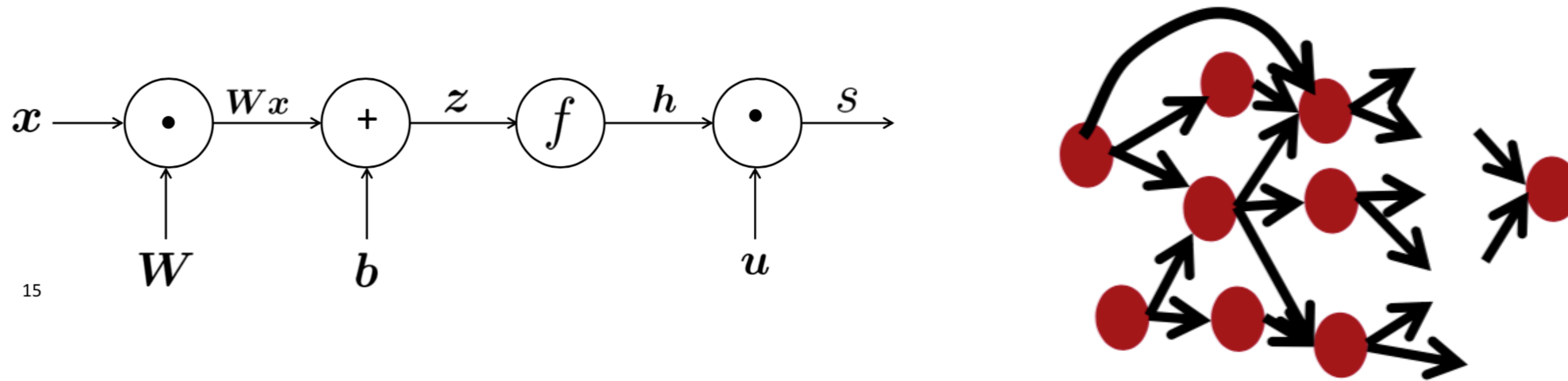
$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W g(V f(\mathbf{x})))$$



$$\frac{\partial \mathcal{L}}{\partial V} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial V}$$

Computation Graphs

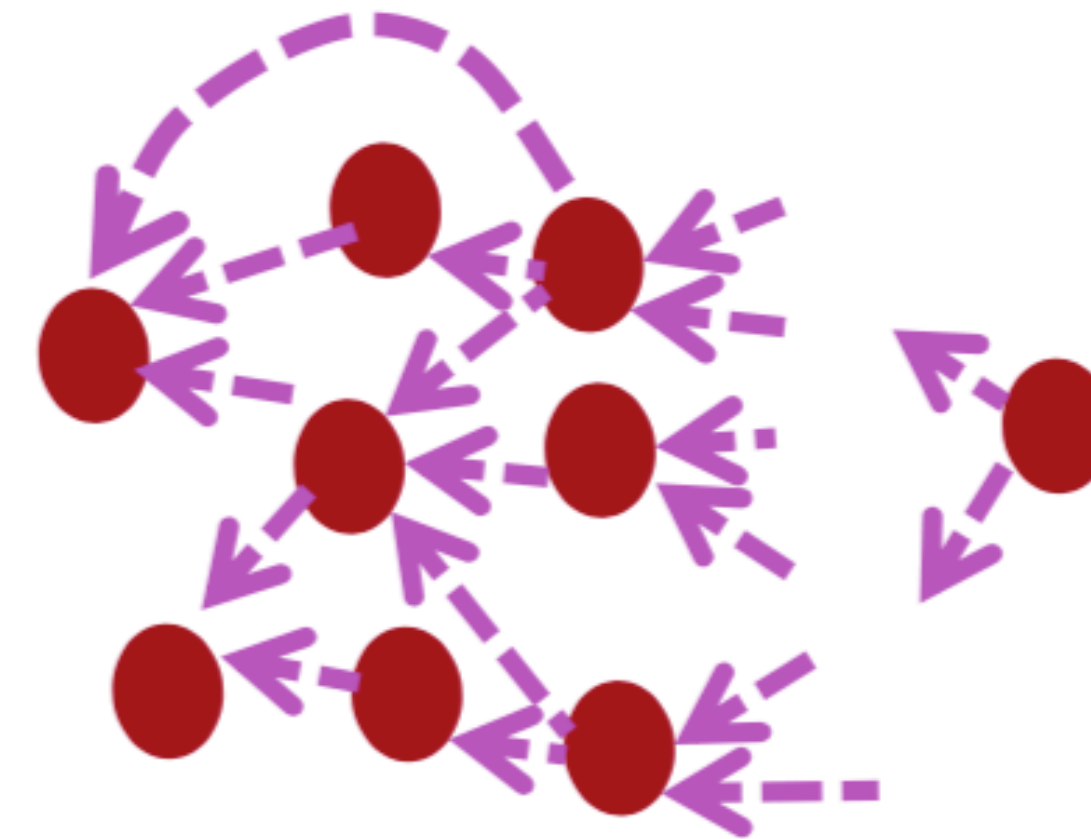
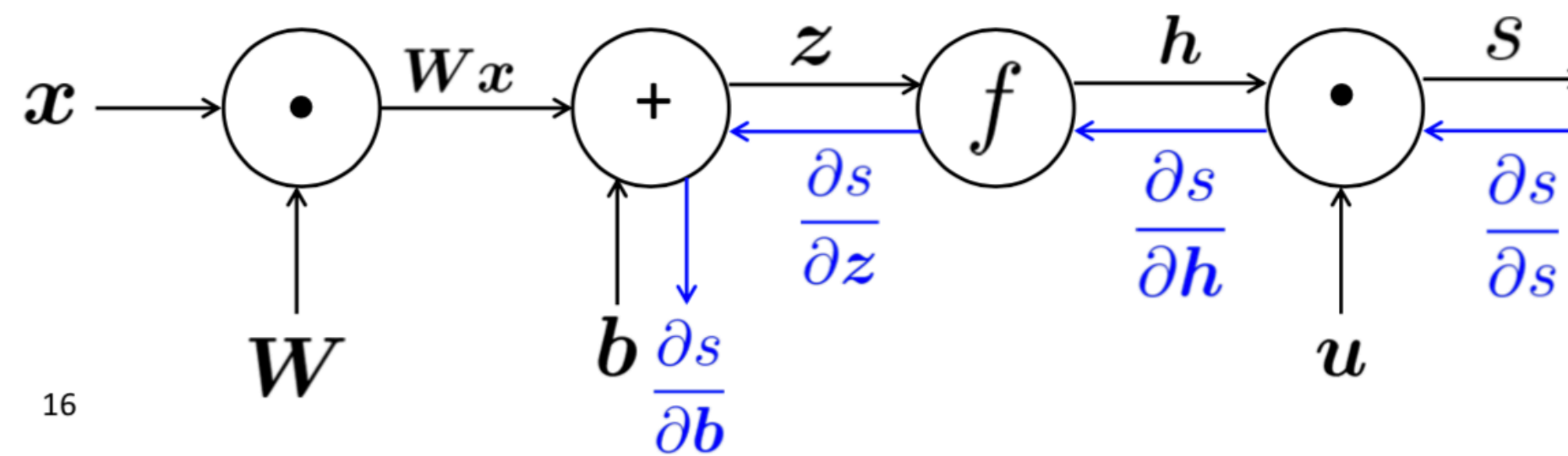
A computation graph describes the function being computed:



- ▶ Given parameters and input, make a prediction
- ▶ Forward pass: visit nodes in topological order

Computational Graphs

Backward computation



- ▶ Loop over the nodes in **reverse** topological order, starting from a final goal node (often our loss function)
- ▶ How does the output change if I make changes to the input?

Backpropagation

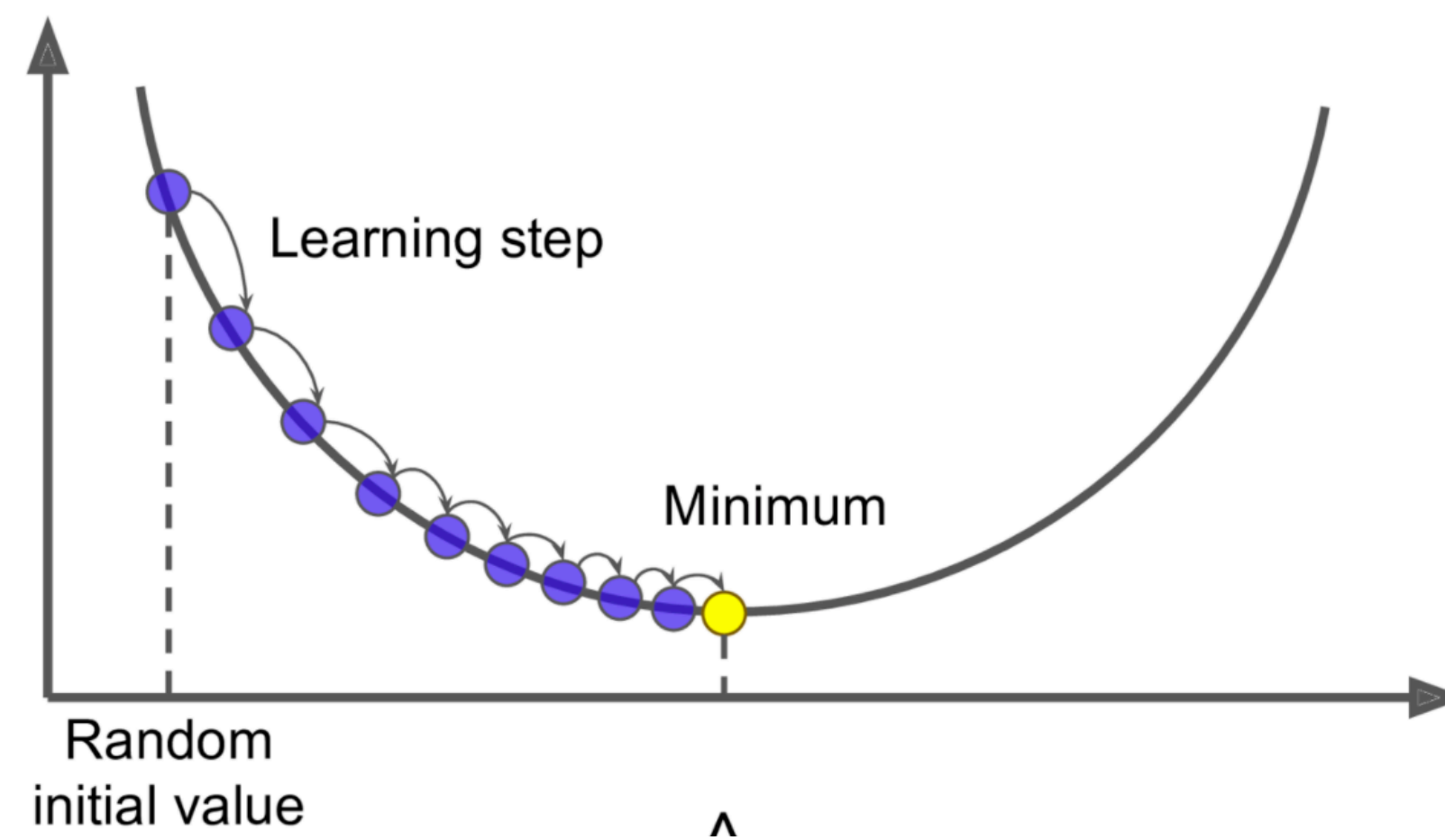
- ▶ Key idea 1: Use the chain rule

$$\frac{\partial}{\partial x} f(g(x)) = f'(g(x))g'(x)$$

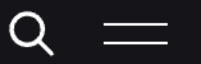
- ▶ Key Idea 2: **Re-using** derivatives computed from the later layers in computing derivations for lower layers, allowing efficient computation of gradients

Recap: Gradient Descent

$$\text{Update: } \theta \leftarrow \theta - \eta \frac{d}{d\theta} L(\theta)$$



ruder.io



optimization

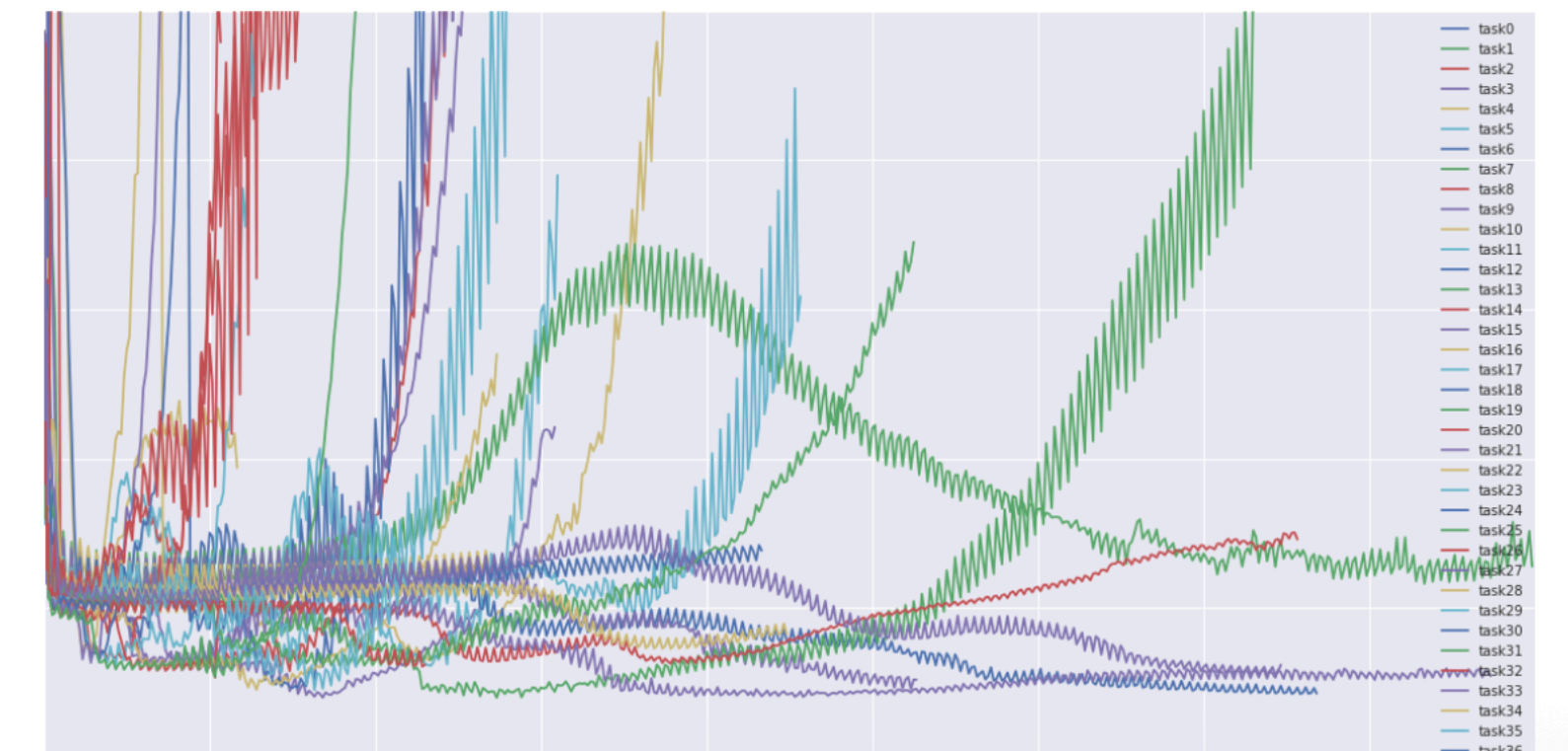
An overview of gradient descent optimization algorithms

Gradient descent is the preferred way to optimize neural networks and many other machine learning algorithms but is often used as a black box. This post explores how many of the most popular gradient-based optimization algorithms such as Momentum, Adagrad, and Adam actually work.



Sebastian Ruder

19 Jan 2016 • 28 min read



Administrative details and recap

Neural Networks and Backpropagation

Word Embeddings

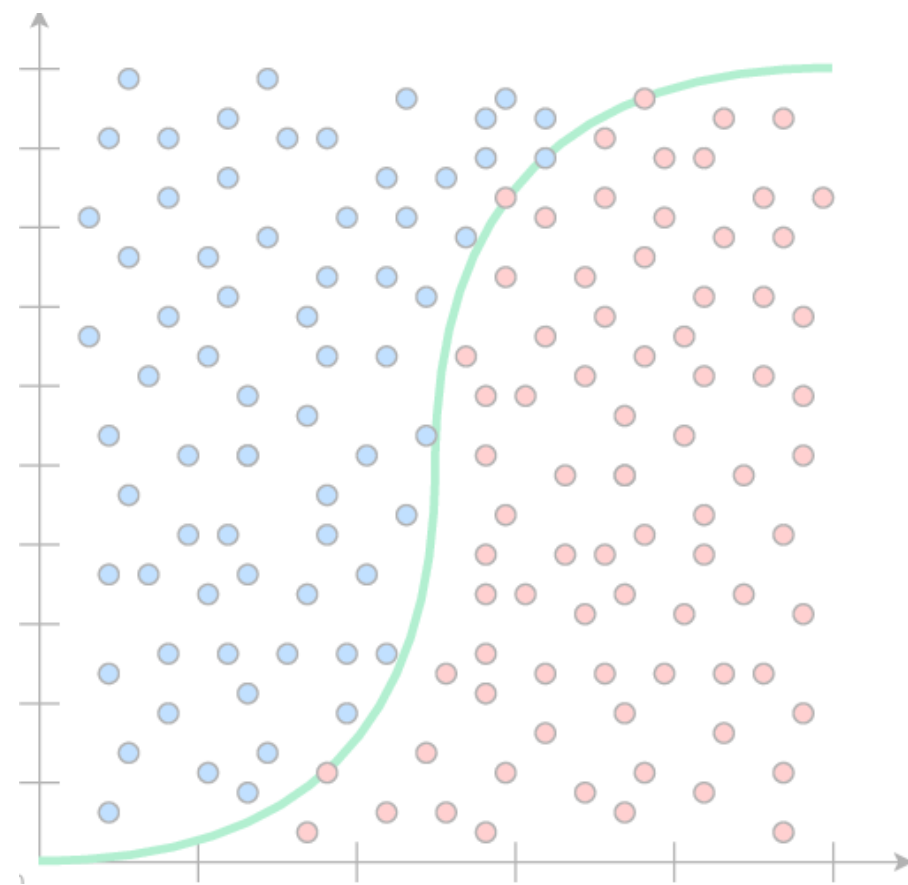
Skip-gram

GloVe

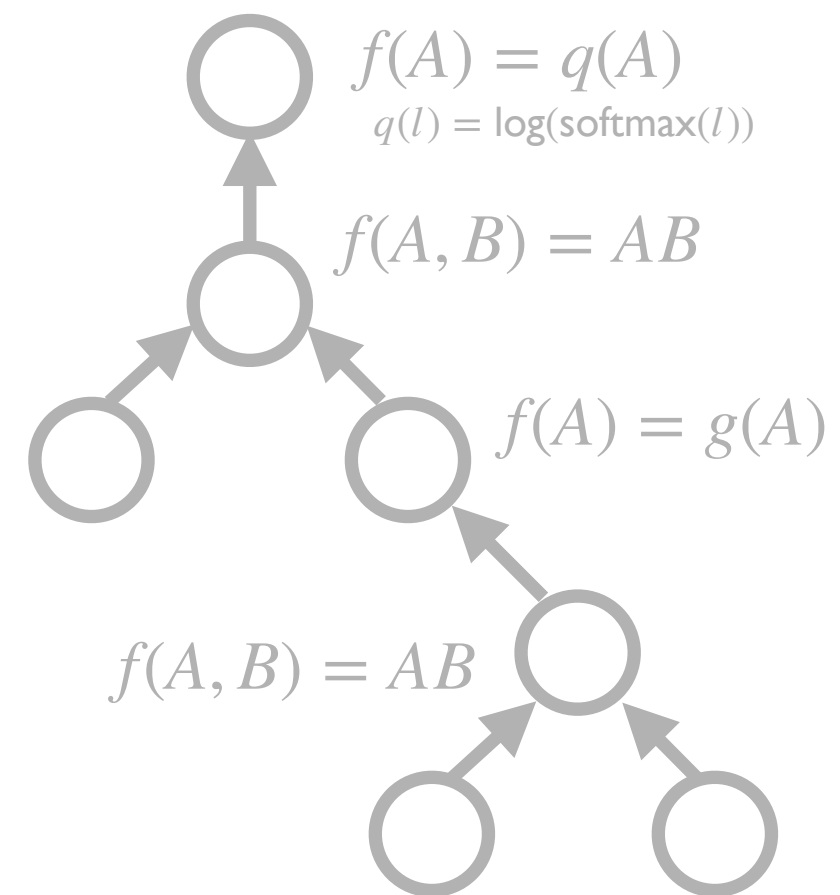
Evaluating Word Embeddings

Word Embeddings

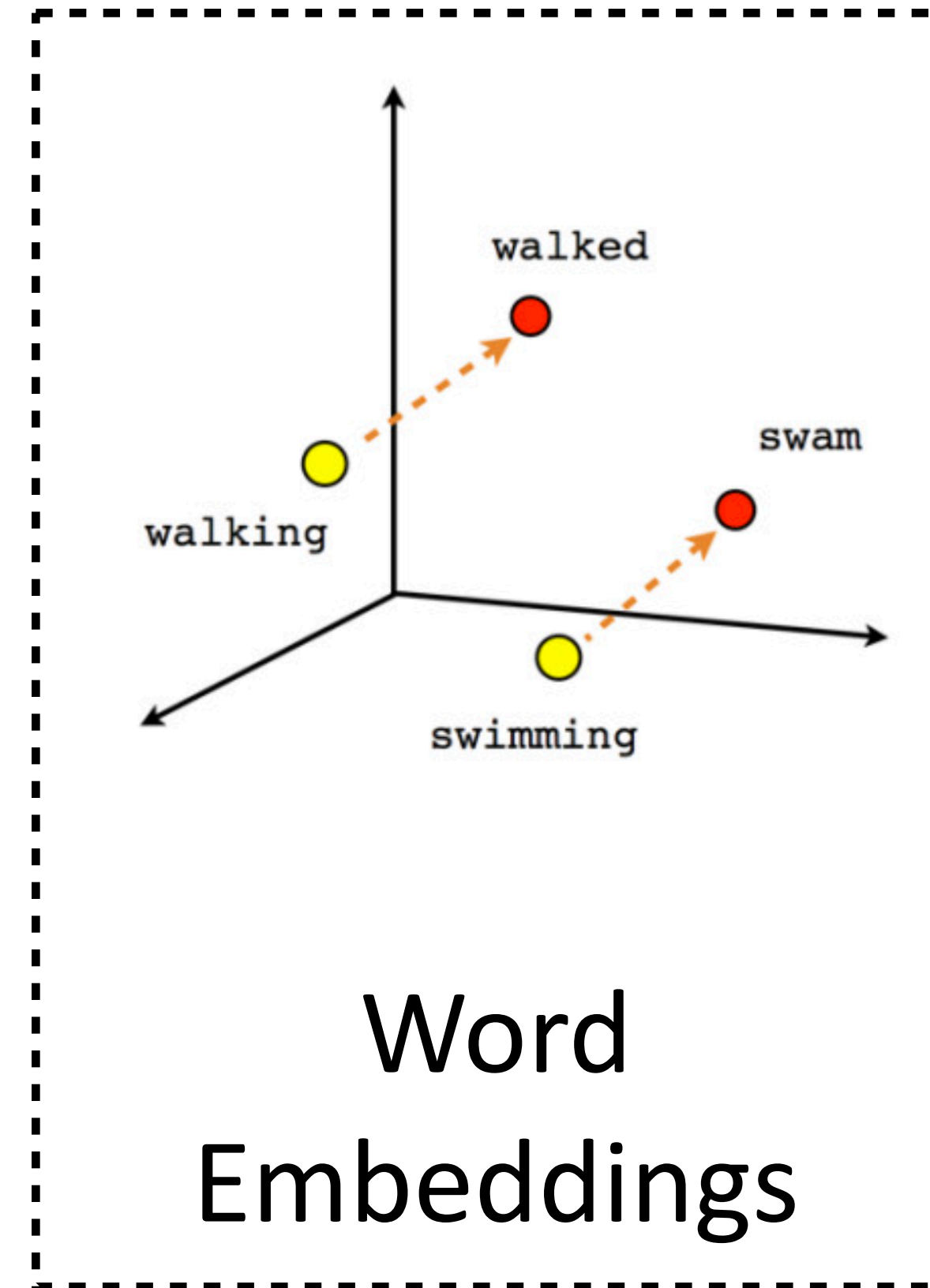
Overview



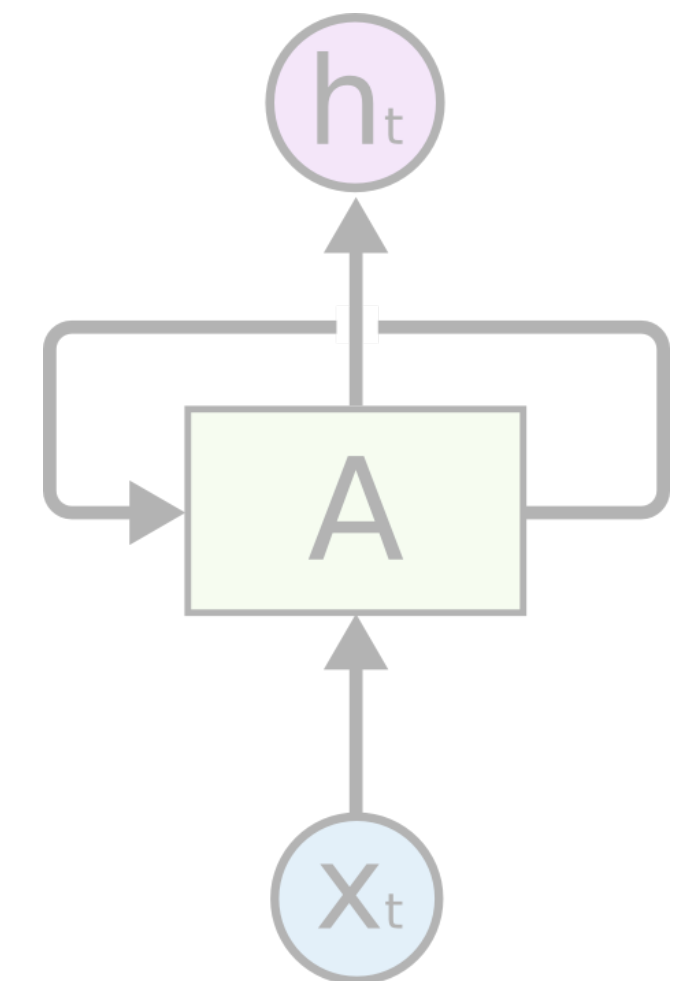
Logistic
Regression



Neural
Networks



Word
Embeddings



Representation of Words

- ▶ Traditional approaches build one-hot feature vectors.

truck = [0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]
austin = [0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0]

- ▶ How big is this vector?
- ▶ No notion of similarity / relationship between words..

car = [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1]
houston = [1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]

Representing Language

- ▶ Neural networks work very well at continuous data, but words are discrete
- ▶ Continuous model $\leftrightarrow$ expects continuous semantics from input

truck, Austin, car, Houston

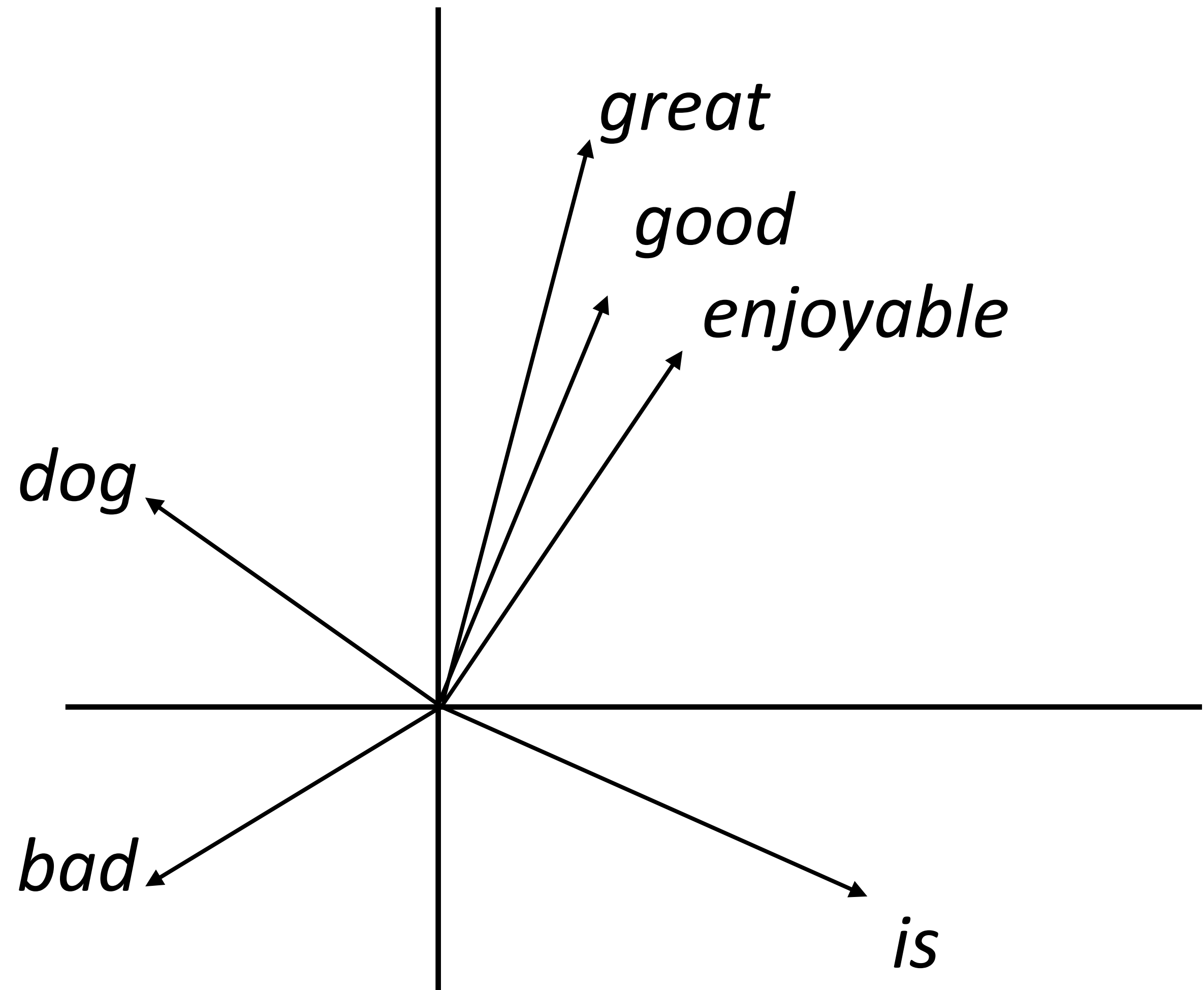
Word Embeddings

the movie was great

$\approx$

the movie was good

- ▶ Goal: come up with a way to produce these embeddings
- ▶ For each word, “medium” dimensional vector (50-300 dims) representing it



Distributional Semantics

“tejuino”



Context 1: A bottle of ____ is on the table.

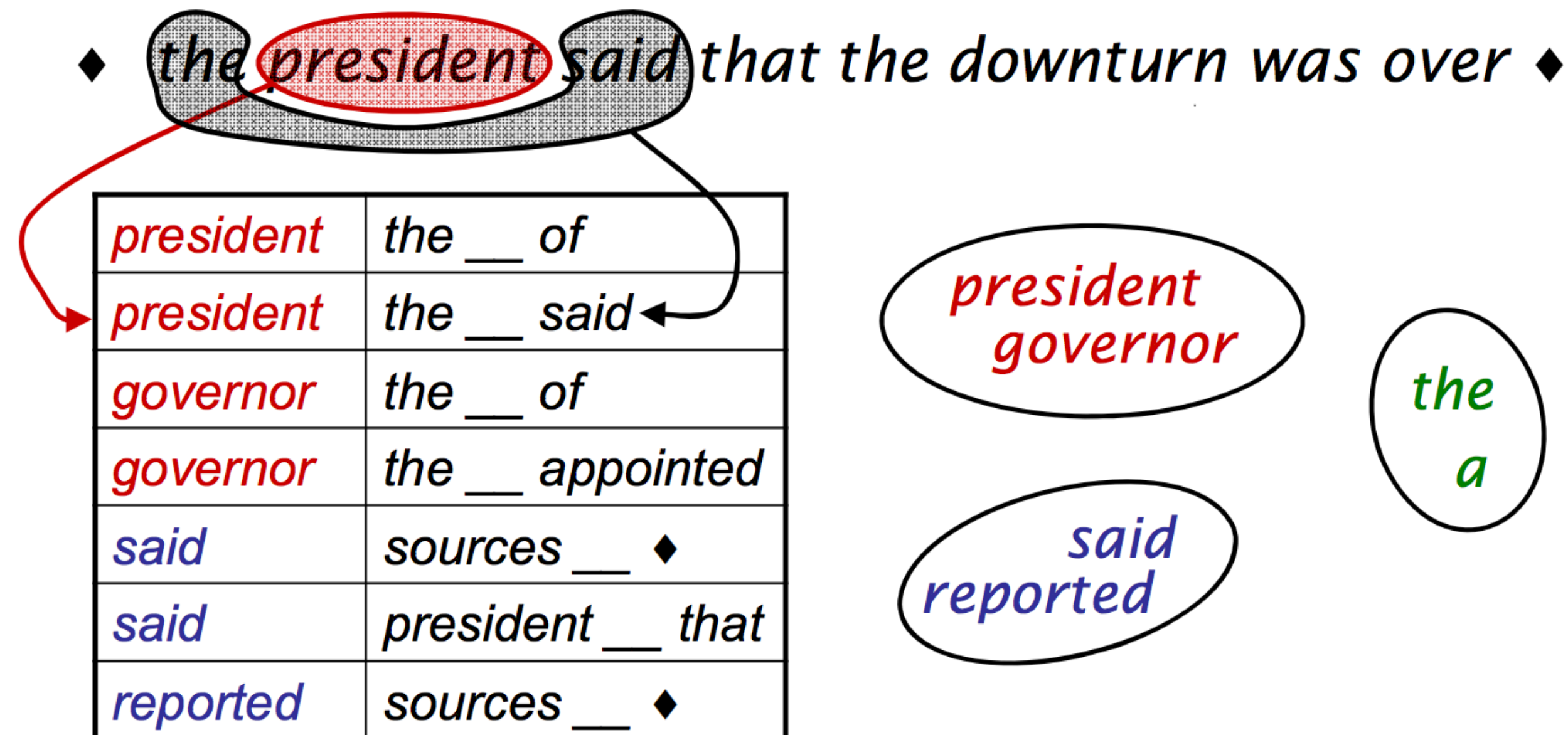
Context 2: Everybody likes ____.

Context 3: Don't have ____ before you drive.

Context 4: We make ____ out of corn.

Distributional Semantics

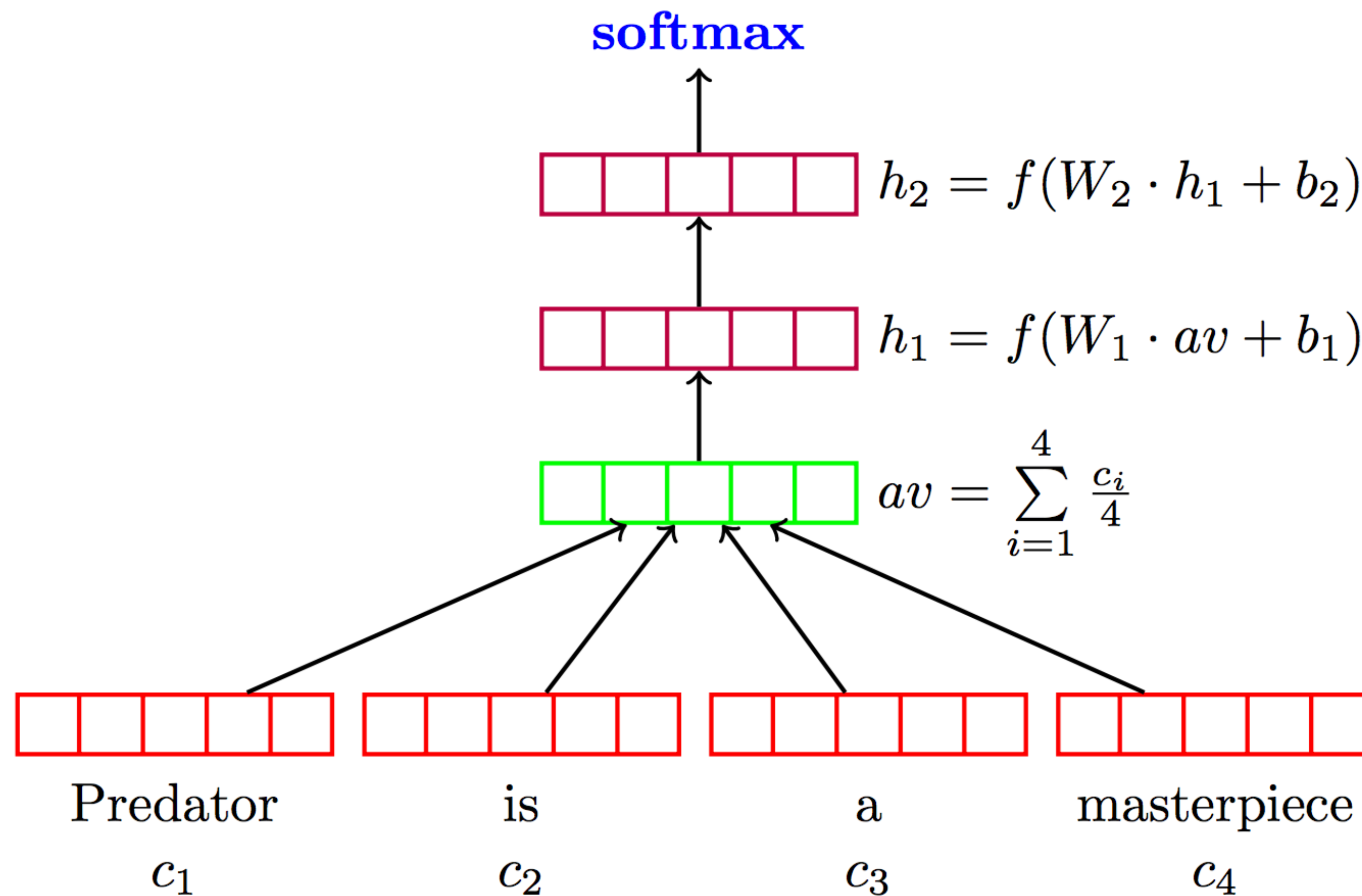
- ▶ “You shall know a word by the company it keeps” Firth (1957)



[Finch and Chater 92, Shuetze 93, many others]

Deep Averaging Network

- Feedforward neural network on average of word embeddings from input



Iyyer et al. (2015)

Administrative details and recap

Neural Networks and Backpropagation

Word Embeddings

Skip-gram

GloVe

Evaluating Word Embeddings

Skip-gram
(see handwritten notes)

Administrative details and recap

Neural Networks and Backpropagation

Word Embeddings

Skip-gram

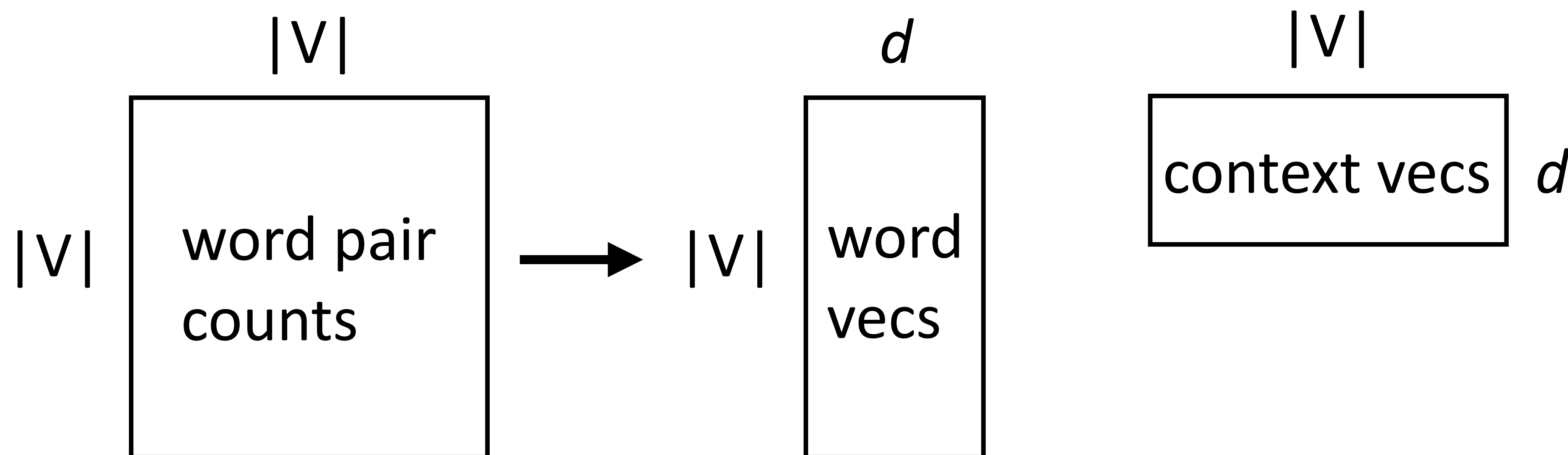
GloVe

Evaluating Word Embeddings

GloVe

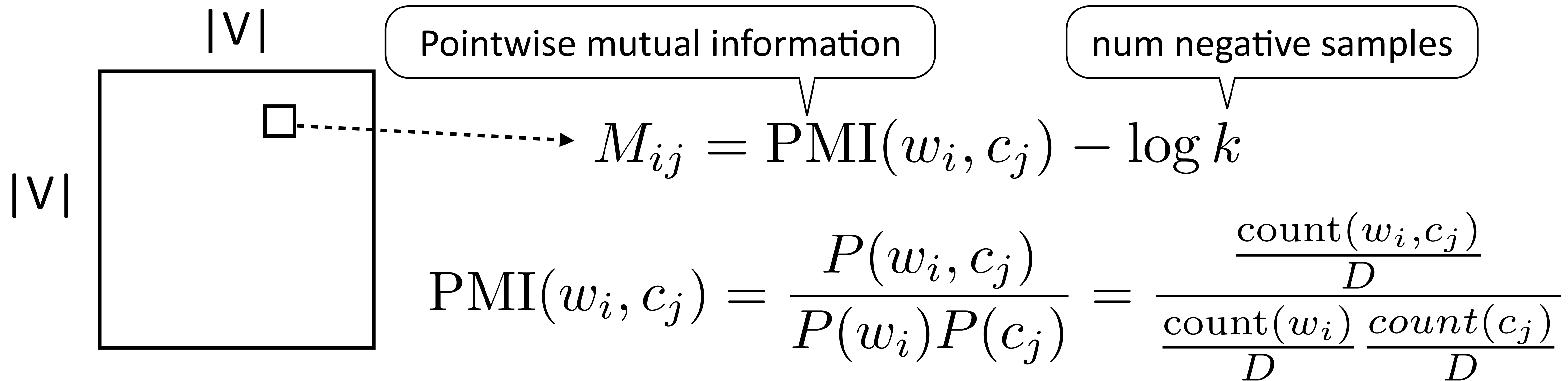
Connections with Matrix Factorization

- ▶ Skip-gram model looks at word-word co-occurrences and produces two types of vectors



- ▶ Looks almost like a matrix factorization... can we interpret it this way?

Skip-Gram as Matrix Factorization

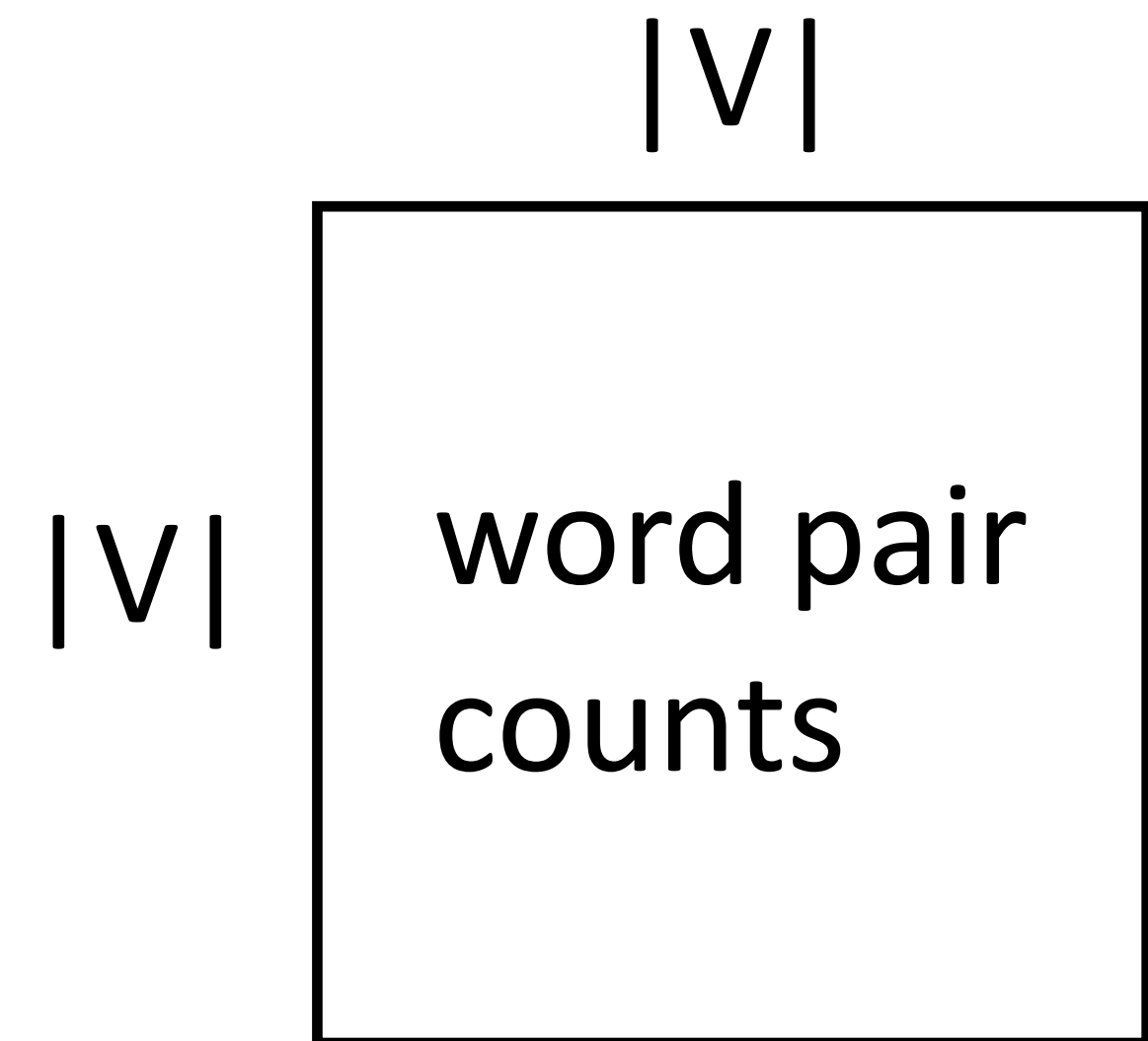


Skip-gram objective *exactly* corresponds to factoring this matrix:

- ▶ If we sample negative examples from the uniform distribution over words
- ▶ ...and it's a *weighted* factorization problem (weighted by word freq)

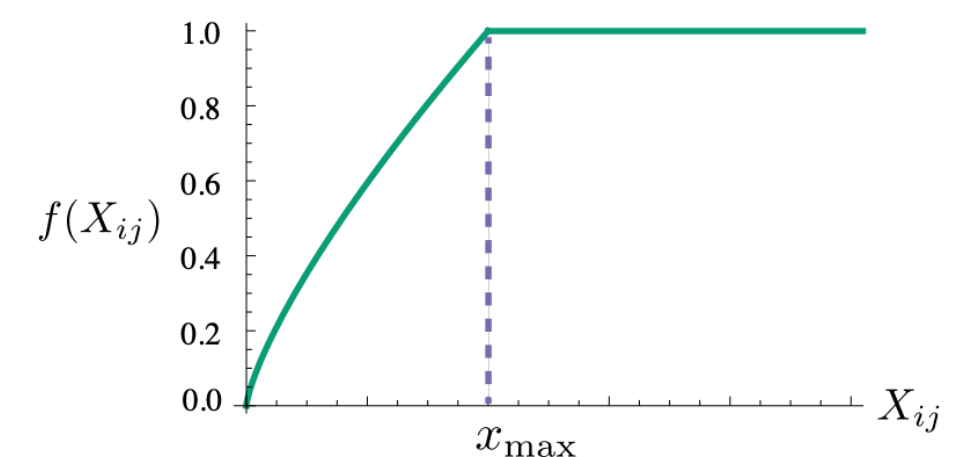
GloVe (Global Vectors)

- ▶ Can we directly capture the ratios of co-occurrence probabilities?
 - ▶ Operates on counts matrix, weighted *regression* on the log co-occurrence matrix



- ▶ Objective =
$$\sum_{i,j} f(\text{count}(w_i, c_j)) \left(w_i^\top c_j + a_i + b_j - \log \text{count}(w_i, c_j) \right)^2$$

- ▶ Constant in the dataset size, quadratic in vocabulary size



A Toy Version of GloVe

*"The quick brown fox jumps over the lazy dog",
"The dog barks and the fox runs",
"Brown dogs and quick foxes are common"*

Our Corpus

A Toy Method for Word Embeddings

"The quick brown fox jumps over the lazy dog",
"The dog barks and the fox runs",
"Brown dogs and quick foxes are common"

Our Corpus

```
sentences = [s.lower().split() for s in lines]
vocab = {w for s in sentences for w in s}
index = {w: i for i, w in enumerate(vocab)}

M = np.zeros((len(vocab), len(vocab)), dtype=float)
for s in sentences:
    for i in range(len(s)):
        for j in range(i + 1, len(s)):
            ia, ib = index[s[i]], index[s[j]]
            if ia != ib:
                M[ia, ib] += 1
                M[ib, ia] += 1

X = PCA(n_components=2).fit_transform(M)

plt.scatter(X[:, 0], X[:, 1])
for i, w in enumerate(vocab):
    plt.annotate(w, (X[i, 0], X[i, 1]))
plt.show()
```

A Toy Method for Word Embeddings

"The quick brown fox jumps over the lazy dog",
"The dog barks and the fox runs",
"Brown dogs and quick foxes are common"

Our Corpus

```
sentences = [s.lower().split() for s in lines]
vocab = {w for s in sentences for w in s}
index = {w: i for i, w in enumerate(vocab)}

M = np.zeros((len(vocab), len(vocab)), dtype=float)
for s in sentences:
    for i in range(len(s)):
        for j in range(i + 1, len(s)):
            ia, ib = index[s[i]], index[s[j]]
            if ia != ib:
                M[ia, ib] += 1
                M[ib, ia] += 1

X = PCA(n_components=2).fit_transform(M)

plt.scatter(X[:, 0], X[:, 1])
for i, w in enumerate(vocab):
    plt.annotate(w, (X[i, 0], X[i, 1]))
plt.show()
```

A Toy Method for Word Embeddings

"The quick brown fox jumps over the lazy dog",
"The dog barks and the fox runs",
"Brown dogs and quick foxes are common"

Our Corpus

```
sentences = [s.lower().split() for s in lines]
vocab = {w for s in sentences for w in s}
index = {w: i for i, w in enumerate(vocab)}

M = np.zeros((len(vocab), len(vocab)), dtype=float)
for s in sentences:
    for i in range(len(s)):
        for j in range(i + 1, len(s)):
            ia, ib = index[s[i]], index[s[j]]
            if ia != ib:
                M[ia, ib] += 1
                M[ib, ia] += 1
```

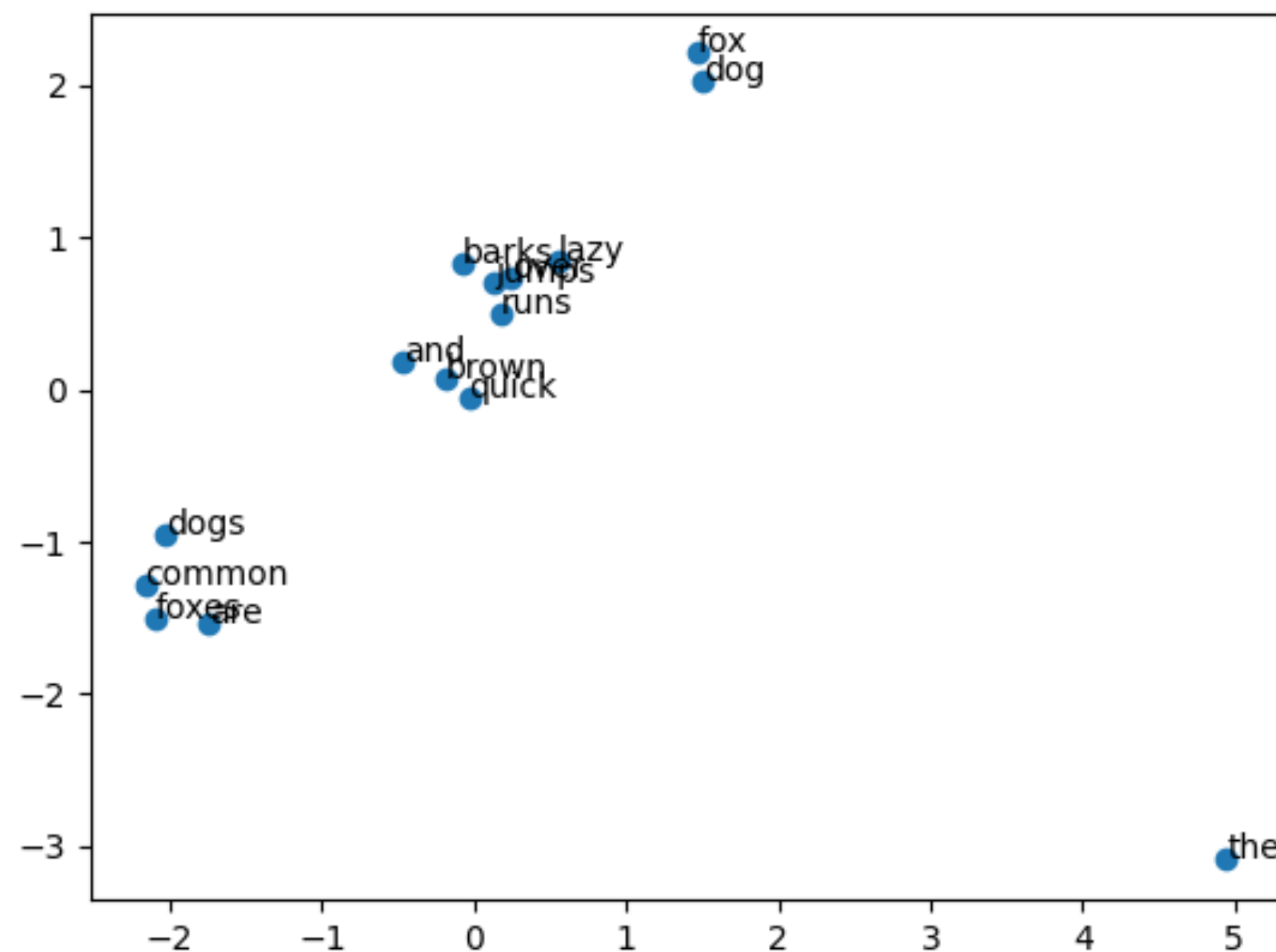
```
X = PCA(n_components=2).fit_transform(M)
```

```
plt.scatter(X[:, 0], X[:, 1])
for i, w in enumerate(vocab):
    plt.annotate(w, (X[i, 0], X[i, 1]))
plt.show()
```

A Toy Method for Word Embeddings

"The quick brown fox jumps over the lazy dog",
"The dog barks and the fox runs",
"Brown dogs and quick foxes are common"

Our Corpus



```
sentences = [s.lower().split() for s in lines]
vocab = {w for s in sentences for w in s}
index = {w: i for i, w in enumerate(vocab)}

M = np.zeros((len(vocab), len(vocab)), dtype=float)
for s in sentences:
    for i in range(len(s)):
        for j in range(i + 1, len(s)):
            ia, ib = index[s[i]], index[s[j]]
            if ia != ib:
                M[ia, ib] += 1
                M[ib, ia] += 1

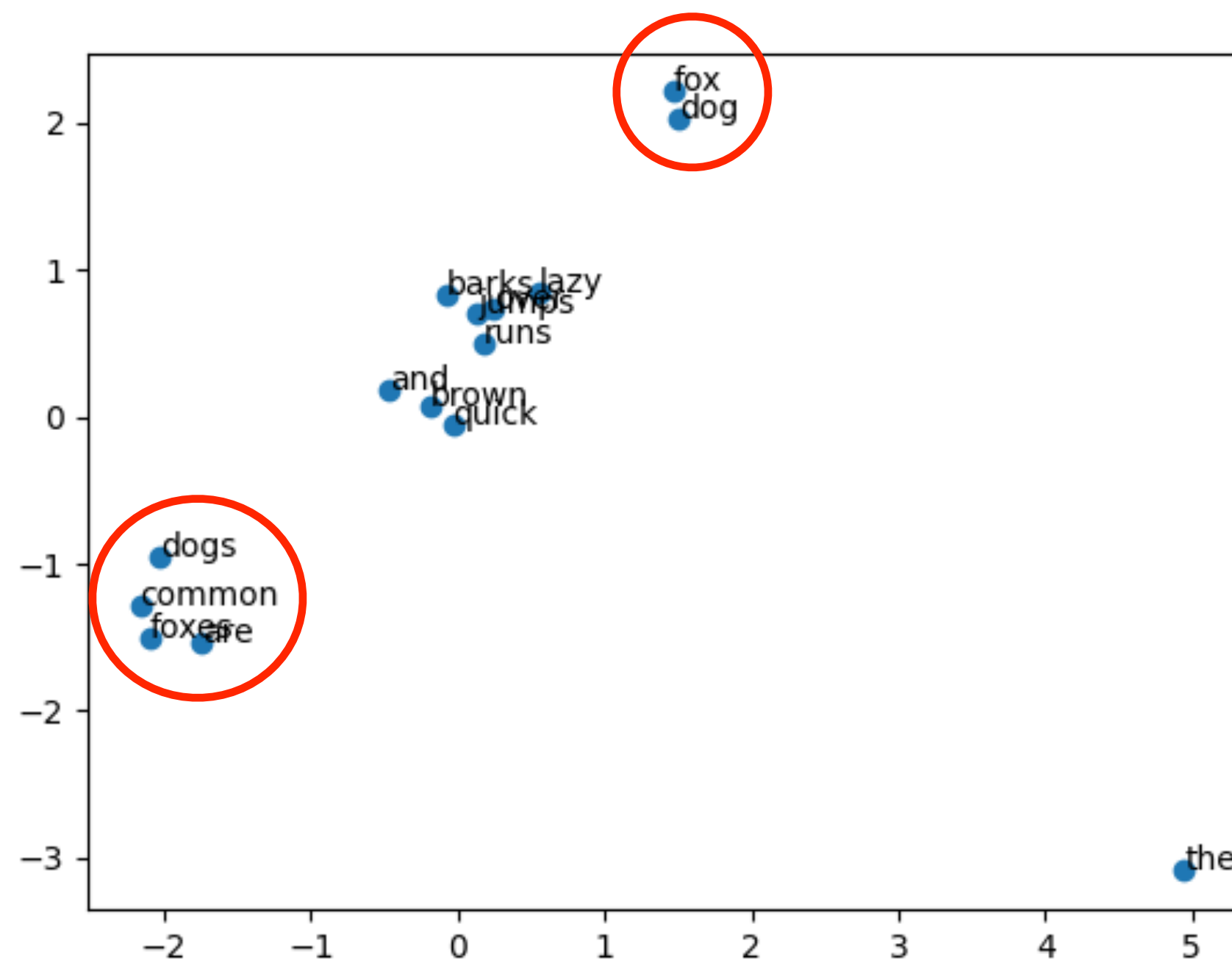
X = PCA(n_components=2).fit_transform(M)

plt.scatter(X[:, 0], X[:, 1])
for i, w in enumerate(vocab):
    plt.annotate(w, (X[i, 0], X[i, 1]))
plt.show()
```

A Toy Method for Word Embeddings

"The quick brown fox jumps over the lazy dog",
"The dog barks and the fox runs",
"Brown dogs and quick foxes are common"

Our Corpus



```
sentences = [s.lower().split() for s in lines]
vocab = {w for s in sentences for w in s}
index = {w: i for i, w in enumerate(vocab)}

M = np.zeros((len(vocab), len(vocab)), dtype=float)
for s in sentences:
    for i in range(len(s)):
        for j in range(i + 1, len(s)):
            ia, ib = index[s[i]], index[s[j]]
            if ia != ib:
                M[ia, ib] += 1
                M[ib, ia] += 1

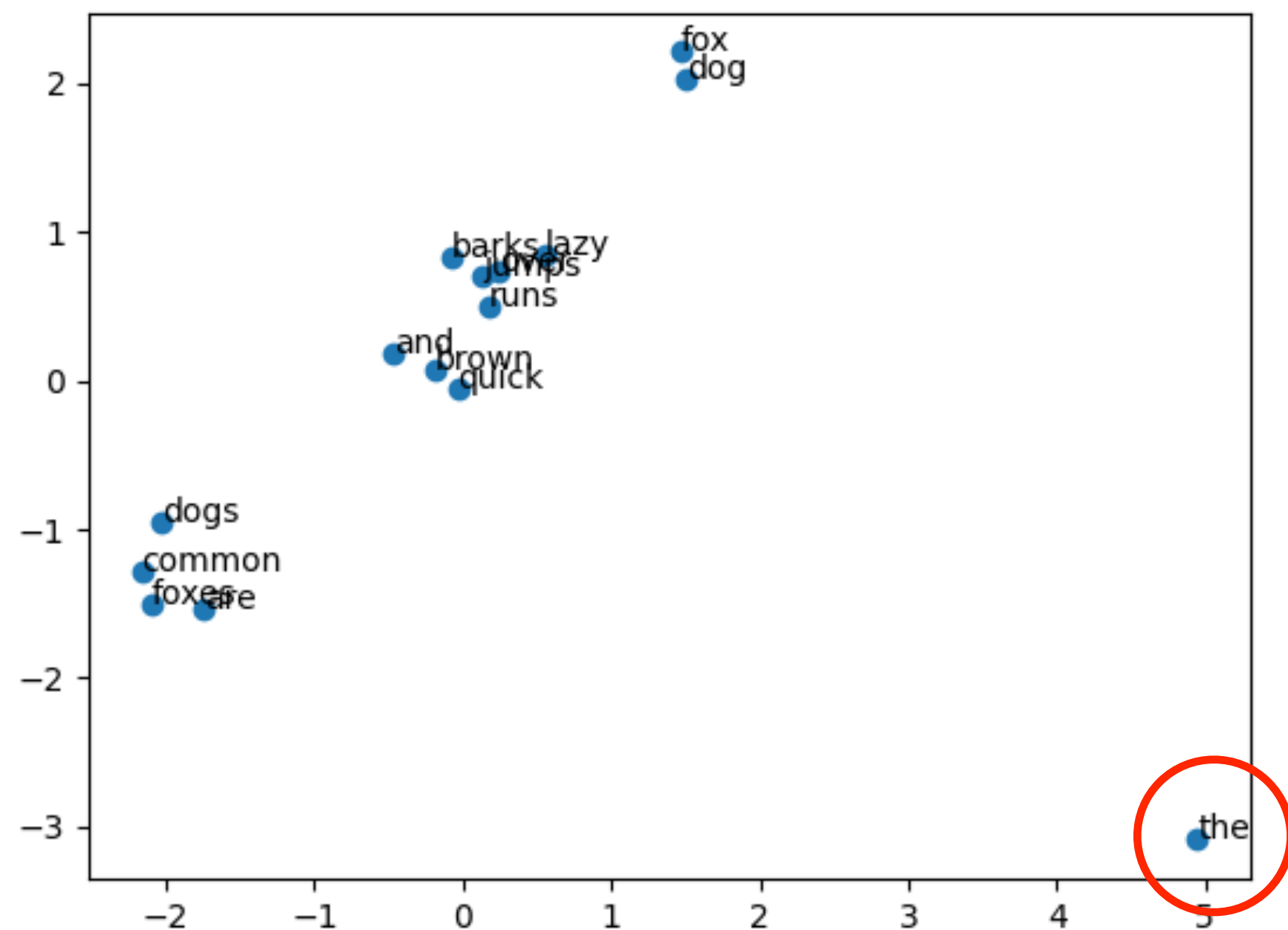
X = PCA(n_components=2).fit_transform(M)

plt.scatter(X[:, 0], X[:, 1])
for i, w in enumerate(vocab):
    plt.annotate(w, (X[i, 0], X[i, 1]))
plt.show()
```

A Toy Method for Word Embeddings

"The quick brown fox jumps over the lazy dog",
"The dog barks and the fox runs",
"Brown dogs and quick foxes are common"

Our Corpus



```
sentences = [s.lower().split() for s in lines]
vocab = {w for s in sentences for w in s}
index = {w: i for i, w in enumerate(vocab)}

M = np.zeros((len(vocab), len(vocab)), dtype=float)
for s in sentences:
    for i in range(len(s)):
        for j in range(i + 1, len(s)):
            ia, ib = index[s[i]], index[s[j]]
            if ia != ib:
                M[ia, ib] += 1
                M[ib, ia] += 1

X = PCA(n_components=2).fit_transform(M)

plt.scatter(X[:, 0], X[:, 1])
for i, w in enumerate(vocab):
    plt.annotate(w, (X[i, 0], X[i, 1]))
plt.show()
```

Administrative details and recap

Neural Networks and Backpropagation

Word Embeddings

Skip-gram

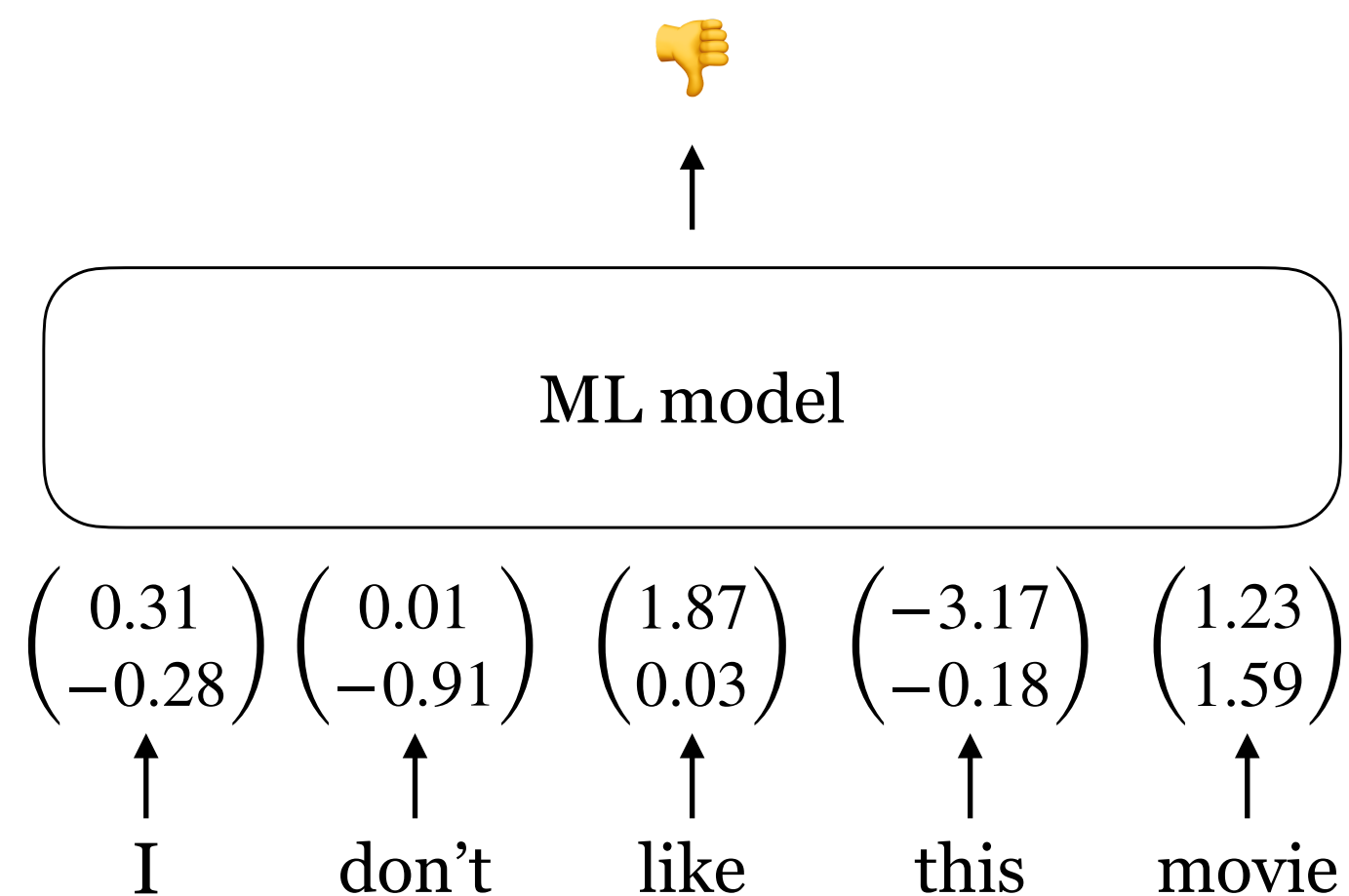
GloVe

Evaluating Word Embeddings

Evaluating Word Embeddings

Extrinsic vs. Intrinsic

- ▶ Plug the vectors into some end-task model, see which does well!
- ▶ Evaluate on some intermediate subtasks (in the coming slides..)



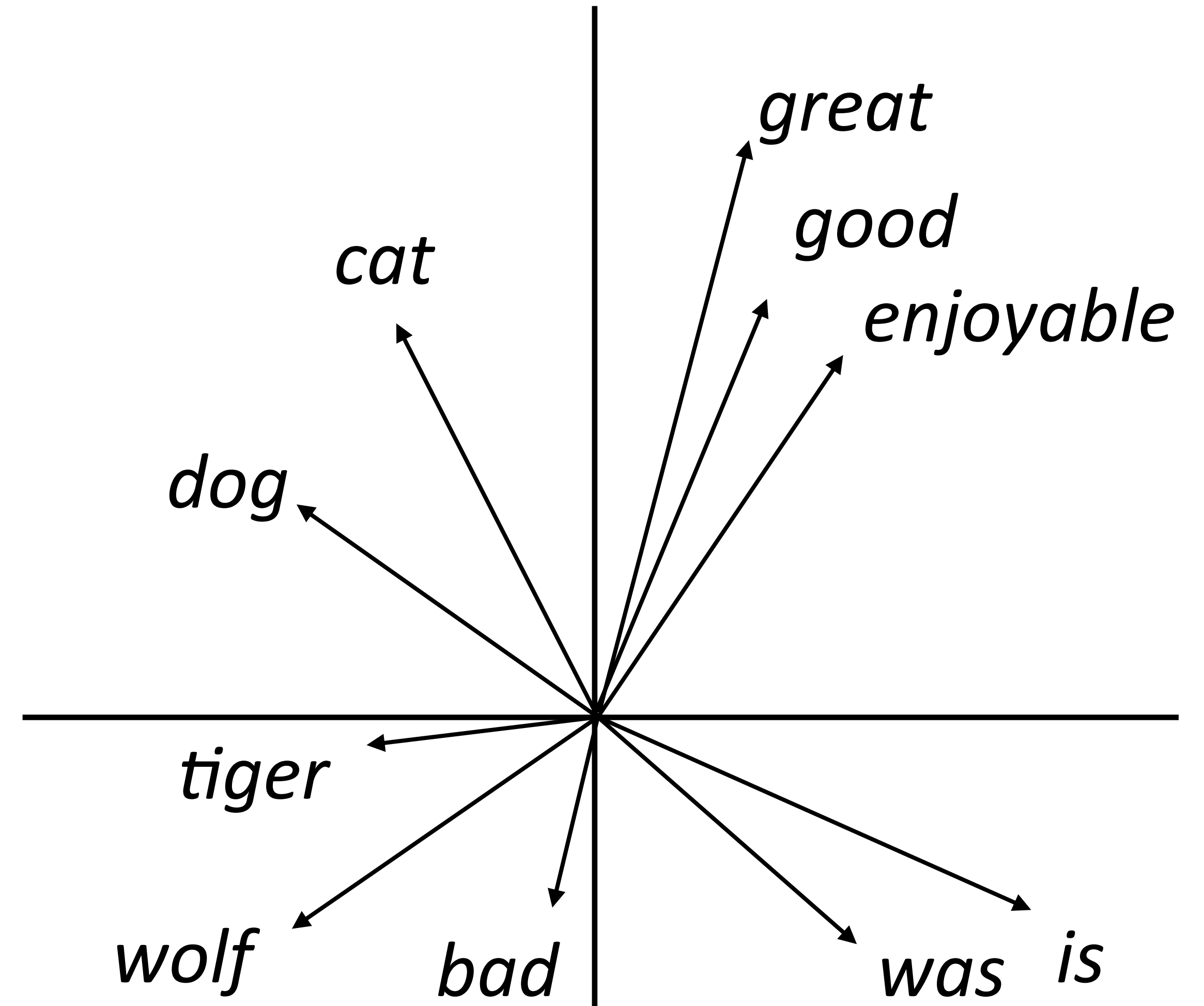
Word 1	Word 2	Human (mean)
tiger	cat	7.35
tiger	tiger	10
book	paper	7.46
computer	internet	7.58
plane	car	5.77
professor	doctor	6.62
stock	phone	1.62
stock	CD	1.31
stock	jaguar	0.92

Evaluating Word Embeddings

- ▶ What properties of language should word embeddings capture?

- ▶ (1) Similarity: similar words are close to each other

Word 1	Word 2	Human (mean)
tiger	cat	7.35
tiger	tiger	10
book	paper	7.46
computer	internet	7.58
plane	car	5.77
professor	doctor	6.62
stock	phone	1.62
stock	CD	1.31
stock	jaguar	0.92



Word Analogies

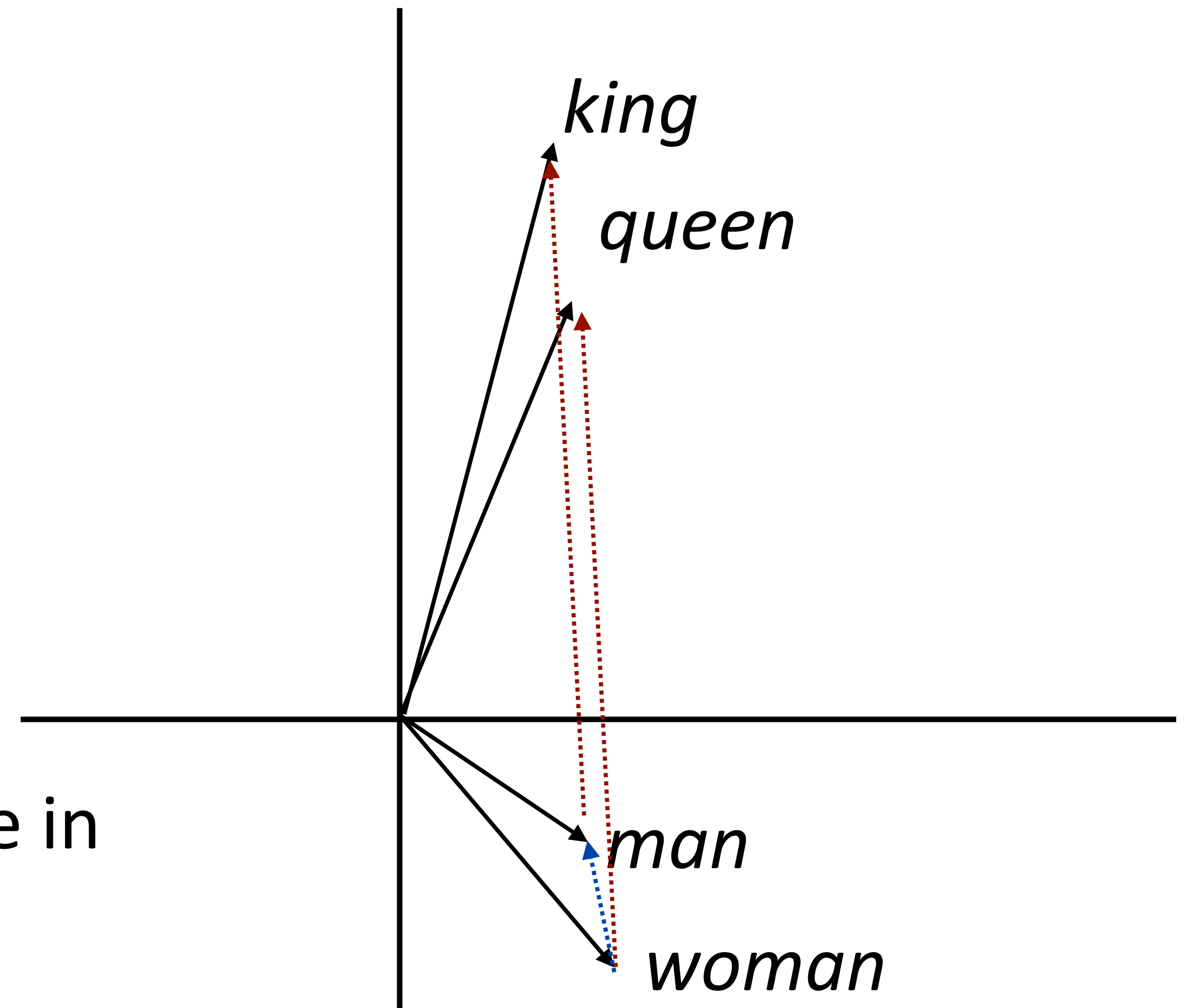
good is to best as smart is to ???

Paris is to France as Tokyo is to ???

(king - man) + woman = queen

king + (woman - man) = queen

- ▶ Why would this be?
- ▶ woman - man captures the difference in the contexts that these occur in



What can go wrong with word embeddings?

- ▶ What's wrong with learning a word's "meaning" from its usage?
- ▶ What data are we learning from?
- ▶ What are we going to learn from this data?

What do we mean by bias?

- Identify *she* - *he* axis in word vector space, project words onto this axis

Extreme *she* occupations

- | | | |
|-----------------|-----------------------|------------------------|
| 1. homemaker | 2. nurse | 3. receptionist |
| 4. librarian | 5. socialite | 6. hairdresser |
| 7. nanny | 8. bookkeeper | 9. stylist |
| 10. housekeeper | 11. interior designer | 12. guidance counselor |

Extreme *he* occupations

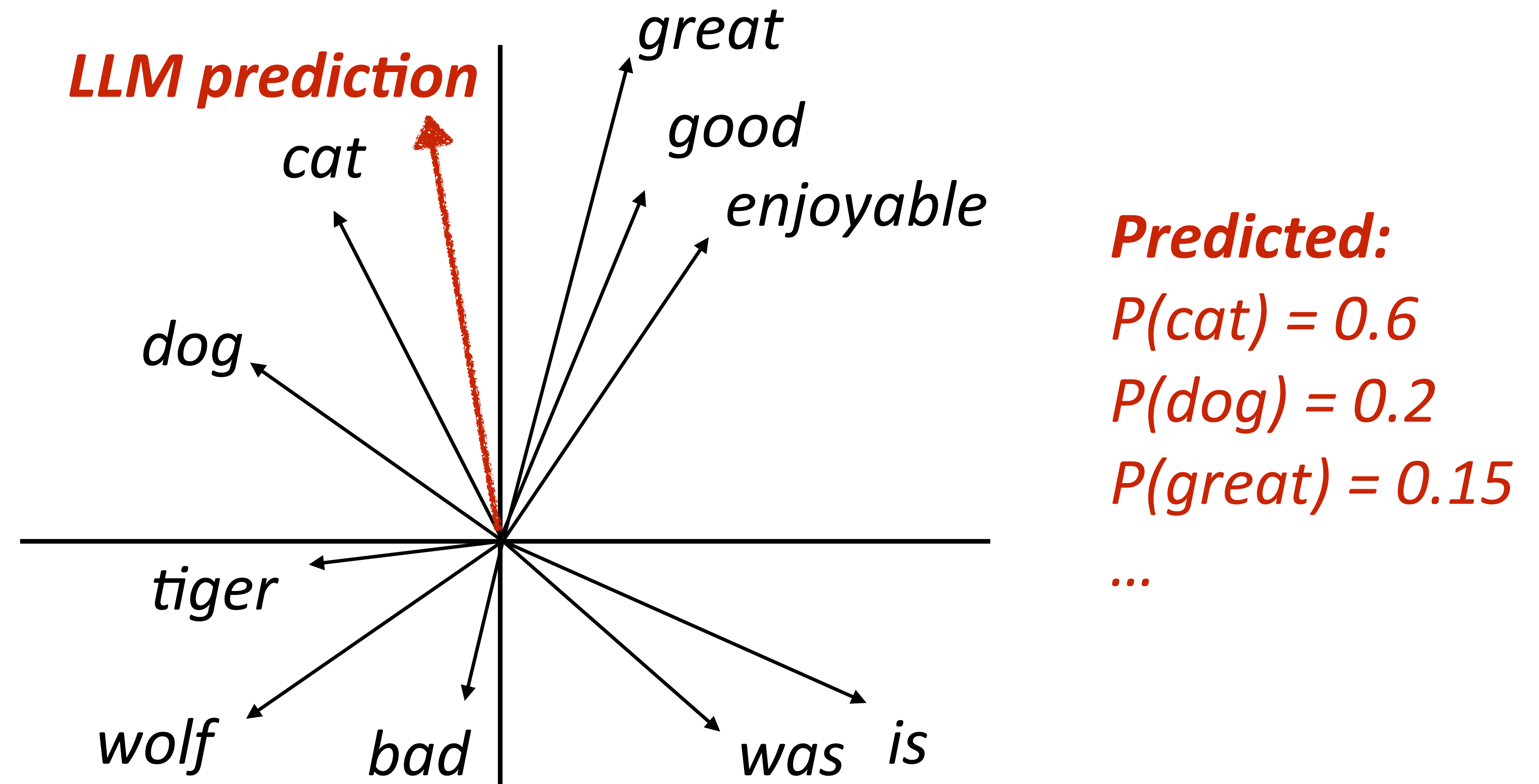
- | | | |
|----------------|-------------------|----------------|
| 1. maestro | 2. skipper | 3. protege |
| 4. philosopher | 5. captain | 6. architect |
| 7. financier | 8. warrior | 9. broadcaster |
| 10. magician | 11. fighter pilot | 12. boss |

Word Embeddings in the Age of LLMs

- ▶ Building a language technology pre-LLMs, for a target task:
 - ▶ Download some pre-trained word embeddings from the Internet
 - ▶ Initialize your language model with these embeddings
 - ▶ Use your task-specific data to finetune the other parameters
- ▶ But we don't do this anymore:
 - ▶ Language models still learn embeddings specific to each word type. The first layer of any Transformer LLM looks up word embeddings.
 - ▶ But we don't download just download embeddings from the Internet; we download the whole language model

Why still care about word embeddings?

- **Preview:** LLMs make predictions via dot products in embedding space



Why still care about word embeddings?

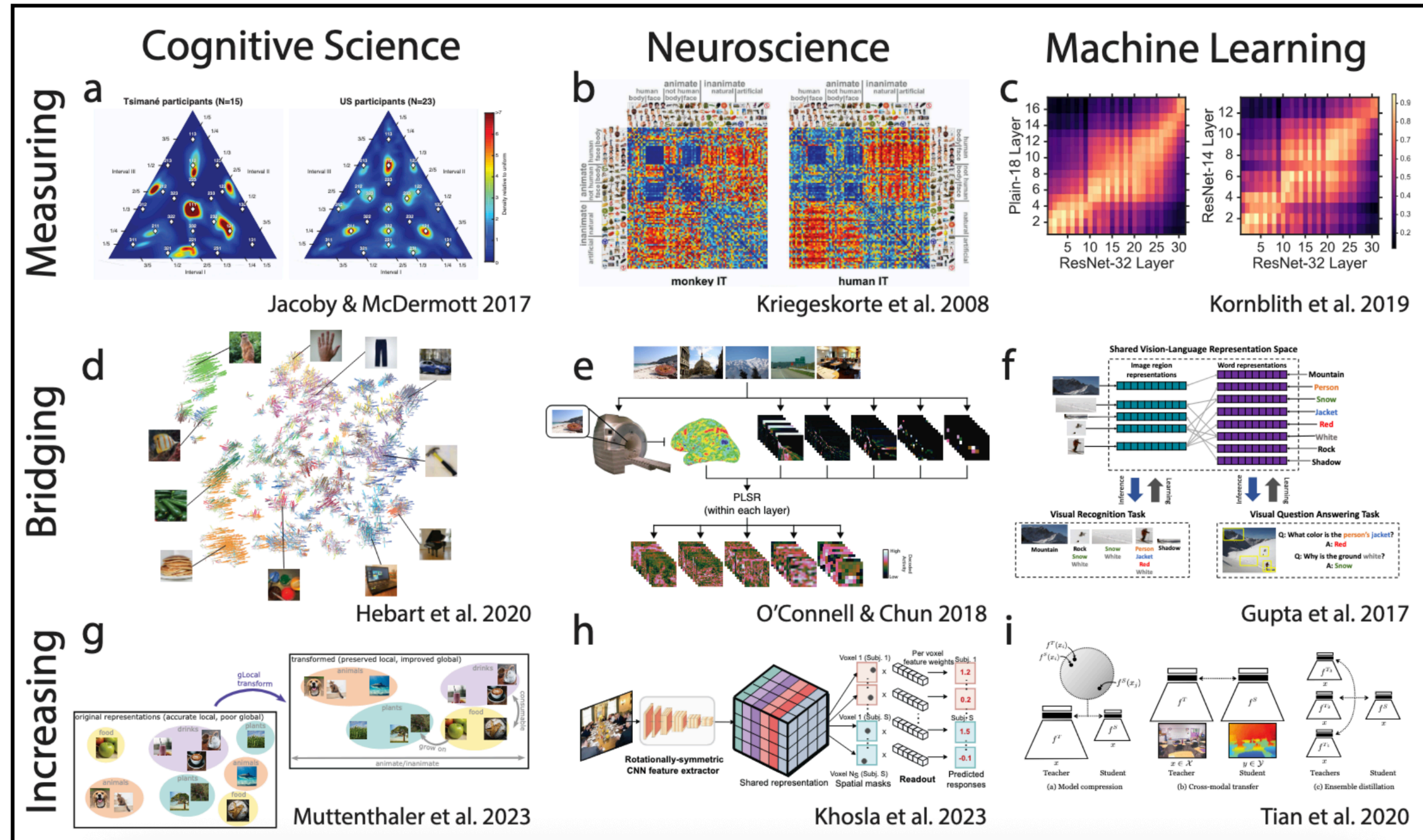
- Any layer of a Transformer is a “prediction”; we can decode its embedding

	(*) , ,	(*) ' we'	' train'	' G'	' PT'	' ,	' 3'	(*) , ,	' an'	' aut'
h_out	' ,	' we'	' show'	' a'	' AN'	' models'	' based'	' ,	' a'	' N'
h46_out	' ,	' we'	' show'	' a'	' AN'	' ,	' L'	' ,	' a'	' N'
h44_out	' ,	' we'	' show'	' a'	' BM'	' models'	' based'	' models'	' a'	' N'

...many layers skipped

h6_out	' focused'	' however'	' "d"	' ees'	' rou'	' ools'	' based'	' rd'	' which'	' enormous'
h4_out	' target'	' however'	' can'	' ees'	' rou'	' ools'	' sided'	' rd'	' and'	' enormous'
h2_out	' guid'	' and'	' Hardy'	' ees'	' rou'	' ools'	' based'	' rd'	' and'	' enormous'
h0_out	' chini'	' and'	' "d"	' train'	' rou'	' ools'	' based'	' rd'	' and'	' isolated'
	' Specifically'	' ,	' we'	' train'	' G'	' PT'	' ,	' 3'	' ,	' an'

Why still care about word embeddings?



Evaluating how well an LLM's learned concept space correlates with human concept spaces

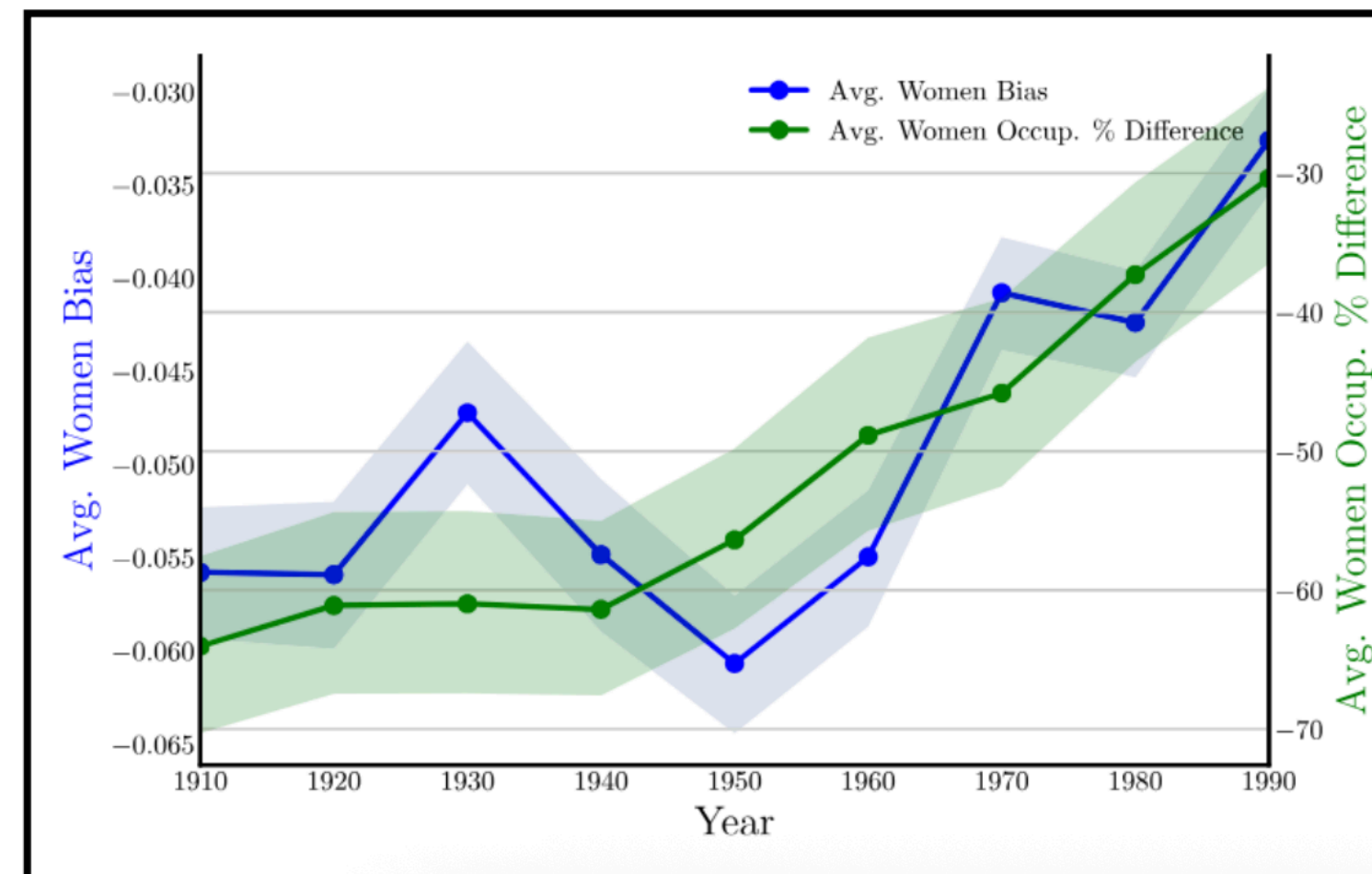
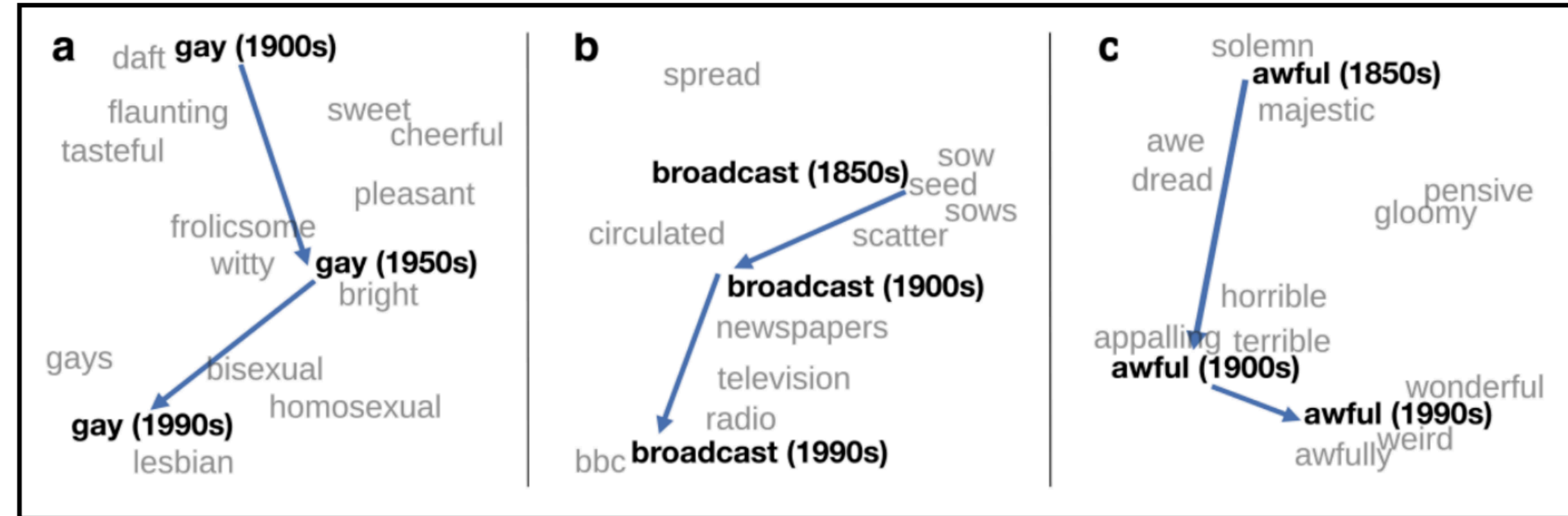
(Sucholutsky et al. 2023)

Why still care about word embeddings?

Analyzing how language changes over time

(Hamilton et al. 2018)

(Garg et al. 2018)



1910	1950	1990
Charming	Delicate	Maternal
Placid	Sweet	Morbid
Delicate	Charming	Artificial
Passionate	Transparent	Physical
Sweet	Placid	Caring
Dreamy	Childish	Emotional
Indulgent	Soft	Protective
Playful	Colorless	Attractive
Mellow	Tasteless	Soft
Sentimental	Agreeable	Tidy

slide credit: Alane Suhr

Administrative details and recap

Neural Networks and Backpropagation

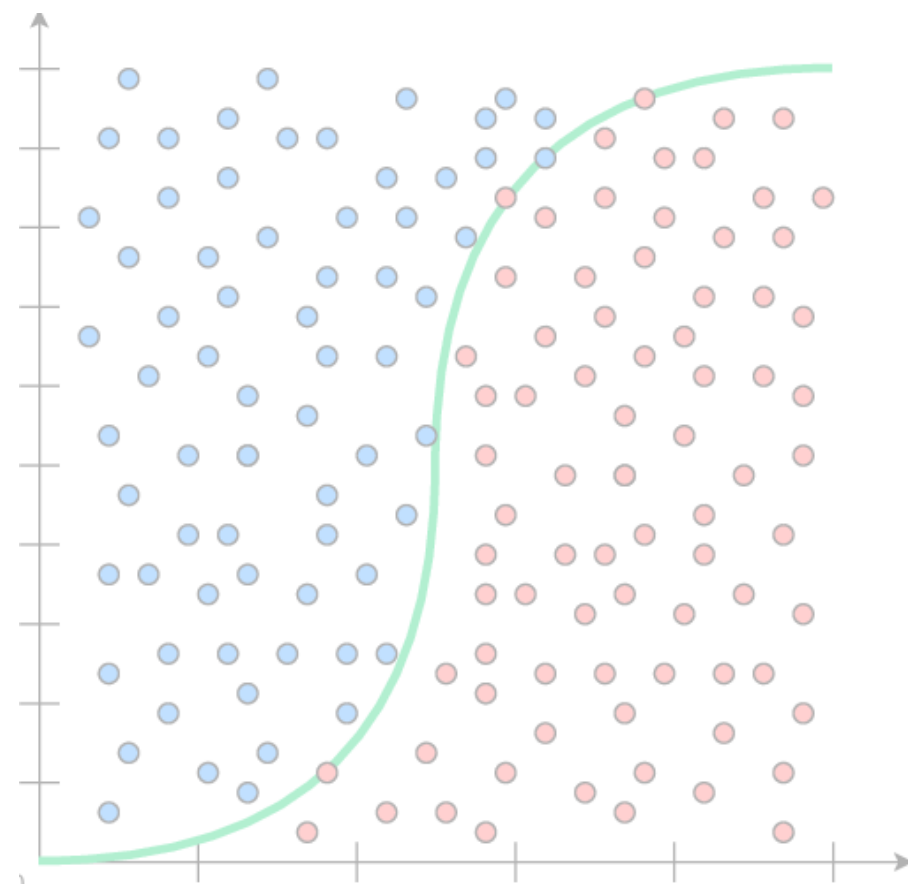
Word Embeddings

Skip-gram

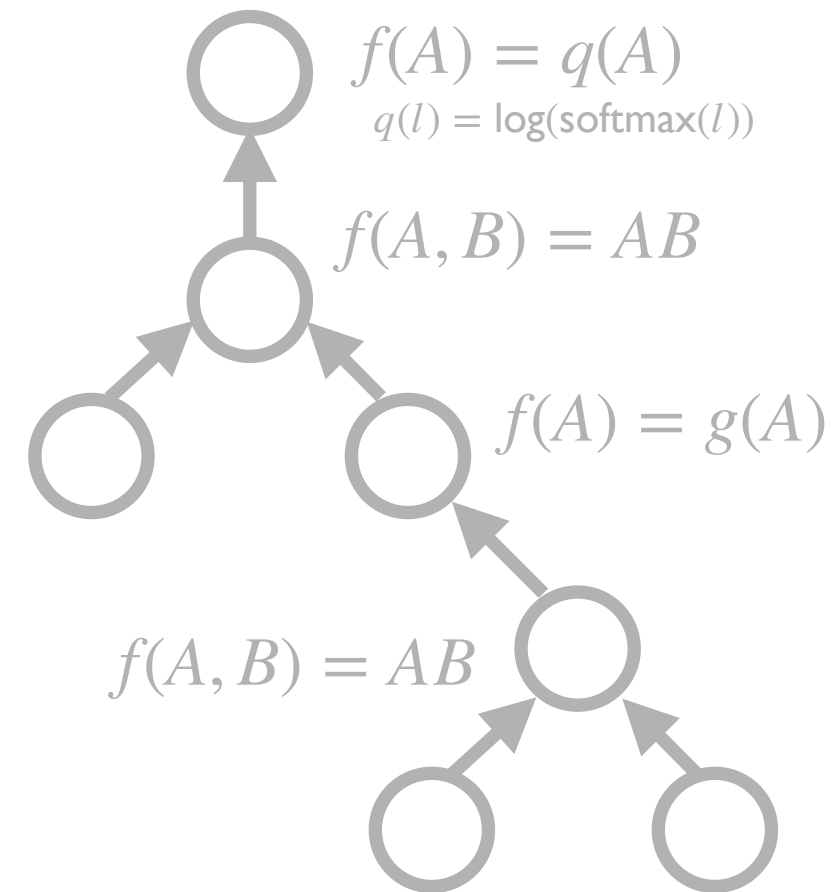
GloVe

Evaluating Word Embeddings

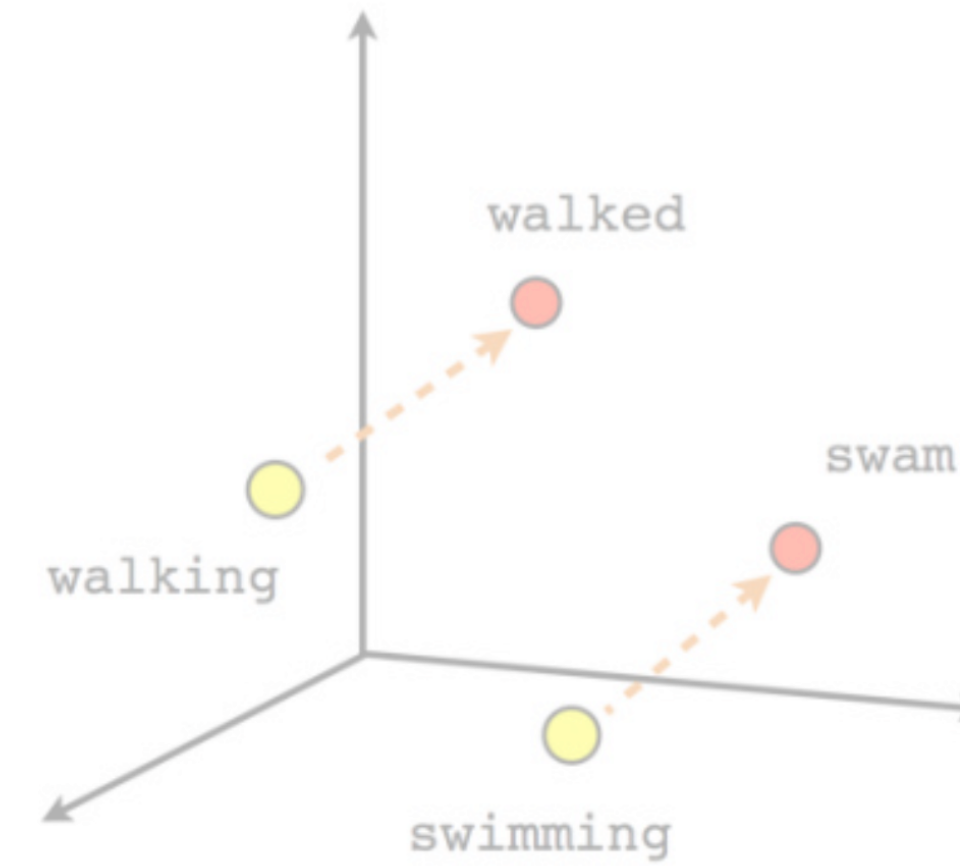
Overview



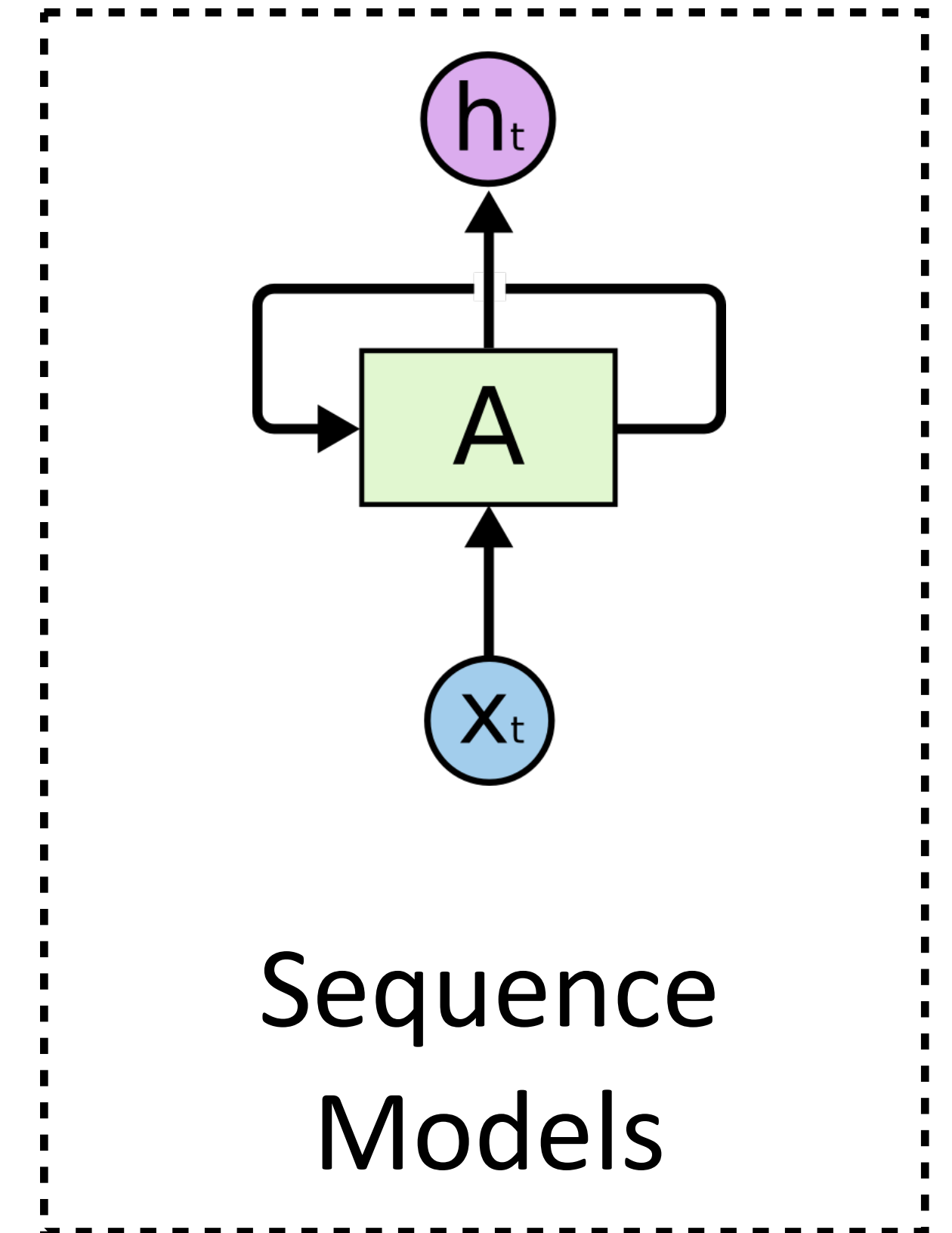
Logistic
Regression



Neural
Networks



Word
Embeddings



Next Week!

Preview: Subwords

What do we do when words haven't been seen before?

He likes to drink tejuino

Subword tokenization:

_He _lik es _to _drink _te jui no

Preview: Context-dependent Embeddings

- ▶ How to handle different word senses? One vector for *bat*

he swings the bat she sees the bat

- ▶ Train a neural language model to predict the next word given previous words in the sentence, use its internal representations as word vectors
- ▶ *Context-sensitive* word embeddings: depend on rest of the sentence
- ▶ *Huge* improvements over static word embeddings. By the second layer of an LLM, embeddings are context-dependent

Compositional Semantics

- ▶ What if we want embedding representations for whole sentences?
- ▶ Skip-*thought* vectors (Kiros et al., 2015), similar to skip-gram generalized to a sentence level
- ▶ Is there a way we can compose vectors to make sentence representations?

Summary

- ▶ Lots of pretrained embeddings work well in practice, they capture some desirable properties
- ▶ Even better: context-sensitive word embeddings (ELMo)
- ▶ Next time: language models!